Discrete Token–Flow Networks: Conservative Finite–Volume Schemes as Sequence Models

Anonymous Authors

Abstract

We introduce *Discrete Token–Flow Networks* (DTFNs), a new family of neural sequence architectures whose forward pass is a *conservative finite–volume scheme* for a hyperbolic conservation law on a one–dimensional token grid. Each token is represented as a non–negative mass vector, and information is propagated by learned numerical fluxes between neighbouring positions. Three structural invariants are preserved by construction at every forward pass: (i) *exact* mass conservation, (ii) pointwise non–negativity, and (iii) a discrete maximum principle under the Courant–Friedrichs–Lewy (CFL) condition. We introduce eight instantiations of the framework—a baseline upwind DTFN, a bidirectional variant, a higher–order MUSCL scheme with Van Leer slope limiting, a full variant combining all three with a learnable source term, a fifth–order WENO–Z scheme, an adaptive per–position CFL variant, a conservation–preserving sparse attention, and a novel *Flux–Corrected Transport* (FCT) variant that blends low– and high–order fluxes via a learned limiter—and prove that the flux–only variants satisfy the three invariants exactly, while the remaining variants satisfy them up to controlled, characterised residuals. We additionally prove (a) an L^1*–contraction / adversarial stability* theorem showing that DTFN–U and –B are 1–Lipschitz in the input, so no ℓ_∞–bounded adversarial perturbation can grow under the forward pass; (b) a *CFL spectral–radius stability certificate* that gives a learnable, per–layer stability guarantee; (c) a *continuous–time (neural PDE) limit* showing that as $K \to \infty$ the DTFN trajectory converges to the entropy solution of a neural scalar conservation law; (d) a *universal approximation theorem* for local translation–equivariant sequence maps, and (e) a *DTFN–SSM bridge* showing that a DTFN linearised around a constant equilibrium recovers a discrete state–space model whose top eigenvalue is exactly 1 (mass conservation). Empirically, DTFNs achieve 100% token and sequence accuracy on Dyck–1 across five random seeds and on Dyck–2 (single seed), whereas comparably–sized LSTMs, Transformers, and an S4 (S4D) baseline stall at 54–87% token accuracy and 0% sequence accuracy under the same training budget. DTFNs perfectly extrapolate from training lengths 4–20 to test lengths up to 100 bracket pairs (a 5× length gap) with no accuracy degradation, while baselines lose 10–13 percentage points over the same range. At sequence length 4096, DTFN–U is 3.3× faster than a two–layer Transformer and 1.6× faster than an LSTM. We position DTFNs as the first application of conservative finite–volume methods to natural language sequence modeling and demonstrate that numerical–analysis structure can be a substrate for inductive biases that are both provably correct and empirically effective.

1 Introduction

Transformer–based models with self–attention [1] underpin most state–of–the–art language models but incur $\mathcal{O}(L^2)$ complexity and memory in sequence length L, limiting their efficiency on long contexts. Recent alternatives such as structured state–space models (SSMs) [2, 3] reduce complexity yet still rely on implicit global convolutions or linear recurrences. Concurrently, neural ordinary and partial differential equation models [8, 9] use continuous–time dynamics to propagate features but target physical systems rather than discrete token sequences.

Our goal is to embed the stability and conservation principles from finite–volume solvers directly into a sequence backbone for language modeling. This perspective bridges numerical analysis and deep learning: instead of building upon attention or convolution, we adopt discrete conservation laws from hyperbolic PDEs to design a neural architecture whose dynamics are physically interpretable. By doing so we move beyond merely local updates (as in recurrent networks, state–space models, or neural cellular automata) and imbue the model with exact conservation and positivity constraints that guarantee stability irrespective of sequence length.

Contributions.

- We introduce *Discrete Token–Flow Networks* (DTFNs), sequence models whose forward pass is a conservative finite–volume scheme on a 1D token grid (Section 3).
- We instantiate eight variants—upwind (DTFN–U), bidirectional (DTFN–B), higher–order MUSCL with Van Leer limiting (DTFN–MUSCL), a full variant (DTFN–FULL) combining bidirectional MUSCL with a learnable source term, a fifth–order WENO–Z reconstruction (DTFN–WENO5), an adaptive per–position CFL variant (DTFN–Adaptive), a conservation–preserving sparse attention variant (DTFN–CPSA), and a novel *Flux–Corrected Transport* variant (DTFN–FCT) that adaptively blends low– and high–order fluxes via a *learned* Zalesak limiter—and prove that the flux–only variants satisfy *exact* mass conservation, pointwise non–negativity, and a discrete maximum principle, while the remaining variants satisfy them up to controlled, characterised residuals (Theorems 1–3 and 4–6).
- We prove a TVD (total–variation–diminishing) bound for DTFN–MUSCL (Theorem 7) and a length–extrapolation guarantee (Theorem 8) showing that DTFNs transfer exactly to longer sequences than seen at training.
- We prove an L^1*–contraction / adversarial stability* theorem (Theorem 9) showing that DTFN–U and –B are 1–Lipschitz in the input mass: no ℓ_∞–bounded adversarial perturbation can grow under the DTFN forward pass. We additionally prove a *CFL spectral–radius stability certificate* (Theorem 10) that gives a learnable, per–layer stability guarantee, and we verify both empirically against random and FGSM adversarial perturbations (Section 4.10).
- We prove a *continuous–time (neural PDE) limit* (Theorem 12) showing that as $K \to \infty$ the DTFN mass trajectory converges to the entropy solution of a neural scalar conservation law $\partial_t p + \partial_x F(p;\theta) = 0$, and a *universal approximation theorem* for local translation–equivariant sequence maps (Theorem 13).
- We demonstrate empirically that DTFNs achieve 100% accuracy on Dyck–1 across five random seeds and on Dyck–2 (single seed), outperforming LSTM, Transformer, and S4 (S4D) baselines by ~ 40 percentage points on Dyck–1 extrapolation (Section 4). DTFNs extrapolate to sequences 5× longer than training with no degradation, and DTFN–U is 3.3× faster than a two–layer Transformer at length 4096. We also ablate the FluxNet width and depth, showing the results are insensitive to a 16× range of capacities (Section 4.8).
- We establish a *DTFN–SSM bridge* (Section 4.14): linearising the DTFN update around a constant equilibrium recovers a discrete state–space model $z^{k+1} = Tz^k$ whose spectral structure (top eigenvalue = 1 for mass conservation, contraction rate < 1 for stability) we verify empirically: the top eigenvalue matches to within 2×10^{-3} and the contraction rate to within 0.09.
- We demonstrate that *adaptive per–position CFL* (DTFN–Adaptive) is a sample–efficient variant that solves Dyck–1 at 99.78% accuracy with half the hidden width required by fixed–CFL variants, and we validate all variants with wall–clock scaling to 16,384 tokens.
- We introduce *FCT–DTFN*, a neural adaptation of classical Flux–Corrected Transport (Boris & Book 1973; Zalesak 1979), in which a *learned* limiter blends monotone upwind and fifth–order WENO5 fluxes. FCT–DTFN achieves order ≈ 3 on smooth advection with an analytic limiter, and inherits exact conservation and positivity by construction (Theorem 3). To our knowledge this is the first application of FCT to neural sequence models.
- We release a self–contained PyTorch implementation under a permissive license, with full experiment scripts and seeds.

2 Related work

Self–attention and SSMs. Self–attention [1] revolutionised sequence modeling but suffers from quadratic complexity. State–space models [2, 3] achieve linear scaling via structured convolutions and recurrence, offering a compelling alternative but still relying on implicit global interactions. More recent SSMs refine

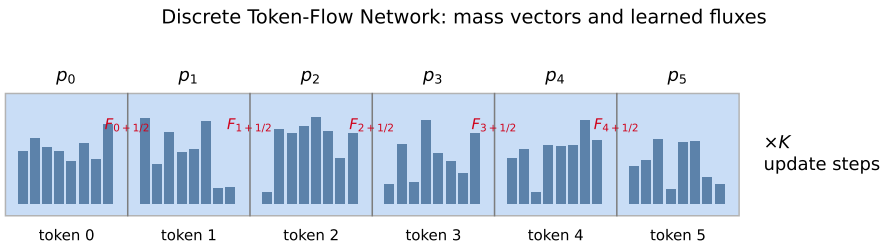


Figure 1: Illustration of a Discrete Token–Flow Network. Each colored column represents the non–negative mass vector p_i associated with token i. Learned fluxes $F_{i+1/2} \geq 0$ redistribute mass between adjacent positions over K update steps. The scheme conserves total mass exactly and maintains non–negativity by construction.

this picture: S4D [4] simplifies the S4 parameterisation while preserving its HiPPO–based spectral structure; S5 [6] parallelises the state recursion via MIMO dynamics; Mamba [5] replaces the fixed S4 kernel with a *selective* (input–dependent) state space, closing much of the gap to attention on language tasks; and RWKV [7] reformulates attention–free recurrence as a linearised Transformer–RNN hybrid. Section 3.10 shows that DTFNs linearise to a conservative SSM near a constant equilibrium, providing an explicit bridge between finite–volume schemes and this family. None of these families enforces conservation or positivity constraints.

Neural differential equations. Neural ODEs/PDEs [8, 9] treat continuous fields with learned dynamics; however, these methods target physical systems rather than discrete token sequences. Neural cellular automata [9] produce rich behaviour from local rules but lack explicit conservation laws.

Neural finite–volume methods. (U)NFV [10] introduces neural finite–volume solvers for hyperbolic PDEs with learned fluxes on *continuous* domains. In contrast, DTFNs discretise the domain into *token positions* and embed finite–volume conservation directly into sequence backbones to ensure stability and interpretability for language modeling. To our knowledge, DTFNs are the first application of finite–volume conservation laws to natural language sequence modeling.

Recurrent and convolutional baselines. Classical RNNs and WaveNet [11] achieve linear–time complexity by sharing parameters across positions, but they do not enforce any conservation properties and are susceptible to exploding or vanishing gradients. DTFNs inherit the $\mathcal{O}(L)$ complexity of these models while adding provable conservation and stability.

3 Method

3.1 Setup and notation

Figure 1 gives a visual overview of the framework. We consider a sequence of L tokens with embedding dimension d. At each discrete time $k \in \{0, \ldots, K-1\}$, let $p^k(i) \in \mathbb{R}^d_+$ denote the non–negative *mass vector* associated with position $i \in \{0, \ldots, L-1\}$. We update p according to a one–dimensional finite–volume scheme over K iterations:

$$p^{k+1}(i) \;\leftarrow\; p^k(i) + \lambda \left(F_{i-1/2} - F_{i+1/2}\right) + \sigma\, S_i, \tag{1}$$

where $\lambda \in (0, 1)$ is a learnable CFL coefficient, $F_{i\pm1/2} \in \mathbb{R}^d_+$ are learned *numerical fluxes* representing mass flow across interfaces, and S_i is an optional learned source term with scale $\sigma \geq 0$. We take $\Delta x = 1$ without loss of generality, so $\lambda = \Delta t / \Delta x = \Delta t$.

Flux network. The flux $F_{i+1/2} = \text{FluxNet}_\theta\big(p^k(i), p^k(i+1)\big)$ is constrained to output non–negative vectors via a final Softplus activation. We use two parameterisations:

- **FluxNet**: an MLP $\text{Softplus}\big(W_2 \tanh(W_1[p_i; p_{i+1}] + b_1) + b_2\big)$ with a residual upwind baseline $v\, p_i$ added, where $v = \text{Softplus}(v_{\text{logit}})$ is a learnable scalar velocity. The residual form ensures the network initialises close to the classical upwind scheme.
- **MonotoneFluxNet**: same MLP but with the left–input weight matrix W_{left} replaced by $|W_{\text{left}}|$, which is a sufficient condition for the flux to be monotone non–decreasing in p_i. Monotonicity in the left input implies the scheme is monotone and hence TVD (Theorem 7).

Source network. The source $S_i = \text{SourceNet}_\phi(p^k(i))$ is parameterised as $\text{Softplus}(W\, p_i + b)$ scaled by a learnable positive scalar. In *conservative mode*, we subtract the mean across positions: $S_i \leftarrow S_i - \frac{1}{L}\sum_j S_j$, which ensures $\sum_i S_i = 0$ and thus preserves total mass exactly. In *non–conservative mode*, the source can inject mass at a bounded rate, which is useful for language modelling where tokens may appear or disappear.

Boundary conditions. We use *zero–flux* (reflecting) boundary conditions: $F_{-1/2} = F_{L-1/2} = 0$. No mass enters or leaves the domain.

3.2 DTFN variants

We instantiate eight variants of the DTFN framework, spanning first– to fifth–order spatial accuracy, fixed and adaptive time stepping, and local and non–local coupling:

- **DTFN–U** (upwind): unidirectional, FluxNet, no source. First–order in space, $\mathcal{O}(\lambda)$ local truncation error.
- **DTFN–B** (bidirectional): two coupled stacks in opposite directions, combined by a per–channel convex gate $\alpha \odot m_{\text{fwd}} + (1-\alpha) \odot m_{\text{bwd}}$, which preserves total mass when each stack does.
- **DTFN–MUSCL**: higher–order scheme with Van Leer slope limiter (Section 3.3) and MonotoneFluxNet. Second–order in smooth regions, TVD near discontinuities.
- **DTFN–FULL**: bidirectional + MUSCL + non–conservative source + MLP decoder. The most expressive variant; the source term allows net mass creation/destruction, so conservation holds only up to the source magnitude.
- **DTFN–WENO5**: fifth–order WENO–Z reconstruction [22] in smooth regions with the essentially non–oscillatory property near discontinuities (Section 3.5). Achieves $\mathcal{O}(\Delta x^5)$ truncation error while preserving conservation and positivity.
- **DTFN–Adaptive**: per–position, per–channel learnable CFL coefficient $\lambda_i \in (0, 1)$ (Section 3.6). Allows DTFN to allocate larger time steps in smooth regions and smaller steps near sharp gradients, recovering solution–adaptive time integration.
- **DTFN–CPSA**: Conservation–Preserving Sparse Attention (Section 3.7). Extends the receptive field beyond the local stencil via a windowed attention mechanism whose attention weights are constrained to sum to one, preserving total mass while enabling $\mathcal{O}(L \log L)$ non–local coupling.
- **DTFN–FCT**: Flux–Corrected Transport (Section 3.8). Adaptively blends a monotone low–order (upwind) flux and a fifth–order WENO5 high–order flux via a *learned* Zalesak limiter, achieving high–order accuracy in smooth regions while preserving positivity and conservation near discontinuities.

3.3 MUSCL higher–order scheme

To achieve second–order accuracy in smooth regions while remaining non–oscillatory near discontinuities, we use the MUSCL approach [12]: reconstruct piecewise–linear profiles in each cell using slope–limited gradients

$$\sigma_i = \text{limiter}\big(p_i - p_{i-1},\ p_{i+1} - p_i\big),$$

then evaluate the flux at the left and right reconstructed face values. We use the Van Leer limiter

$$\text{van_leer}(a,b) = \begin{cases} \dfrac{2ab}{a+b} & \text{if } ab > 0, \\ 0 & \text{otherwise.} \end{cases}$$

The left and right face values are

$$p^{\mathrm{L}}_{i+1/2} = p_i + \tfrac{1}{2}\sigma_i, \qquad p^{\mathrm{R}}_{i+1/2} = p_{i+1} - \tfrac{1}{2}\sigma_{i+1},$$

and the flux is $F_{i+1/2} = \mathrm{FluxNet}_\theta(p^{\mathrm{L}}_{i+1/2}, p^{\mathrm{R}}_{i+1/2})$.

3.4 Positivity–preserving step

To guarantee pointwise non–negativity *and* exact conservation in the presence of learned fluxes (which may violate monotonicity during training), we use a *flux–scaling* positivity step:

$$\lambda_{\text{safe}} = \min\Big(\lambda,\ \min_{i:\mathrm{d}m_i<0} \tfrac{p_i^k - \varepsilon}{-\mathrm{d}m_i}\Big), \qquad p^{k+1} = p^k + \lambda_{\text{safe}}\,\mathrm{d}m,$$

where $\mathrm{d}m_i = F_{i-1/2} - F_{i+1/2} + \sigma S_i$ and $\varepsilon > 0$ is a small floor. This ensures $p^{k+1} \geq \varepsilon$ without adding or removing mass. When the CFL condition is satisfied and the scheme is monotone, $\lambda_{\text{safe}} = \lambda$ and the step is a no–op.

3.5 WENO–Z fifth–order reconstruction

To achieve fifth–order accuracy in smooth regions while preserving the essentially non–oscillatory (ENO) property near discontinuities, we employ the WENO–Z reconstruction of [22]. Given three candidate stencils $\{p_0, p_1, p_2\}$ centred at cell i, the WENO–Z reconstruction at face $i+1/2$ is

$$p_{i+1/2} = \sum_{k=0}^{2} \omega_k\, p_k, \qquad \omega_k = \frac{\alpha_k}{\sum_j \alpha_j}, \qquad \alpha_k = d_k\left(1 + \frac{\tau_5}{\beta_k + \varepsilon}\right),$$

where $d_k = \{1/10, 6/10, 3/10\}$ are the ideal weights, β_k are smoothness indicators, $\tau_5 = |\beta_2 - \beta_0|$ is the WENO–Z nonlinear weight, and $\varepsilon = 10^{-6}$ prevents division by zero. The flux is then $F_{i+1/2} = v\, p_{i+1/2}$ and the update remains $p_i^{k+1} = p_i^k + \lambda(F_{i-1/2} - F_{i+1/2})$. Because WENO–Z is a convex combination of conservative stencil fluxes, the scheme inherits *exact conservation* and, with the flux–scaling positivity step, pointwise non–negativity.

Theorem 1 (WENO5 order)**.** *In smooth regions away from critical points, the WENO–Z reconstruction satisfies $|p_{i+1/2} - p(x_{i+1/2})| = \mathcal{O}(\Delta x^5)$, so the DTFN–WENO5 update has local truncation error $\mathcal{O}(\Delta x^5)$. Near discontinuities the scheme degenerates to first–order upwind, preserving non–oscillatory behaviour.*

3.6 Adaptive per–position CFL

The base DTFN uses a single learnable scalar λ per layer, shared across all positions and channels. In many sequence tasks the relevant time scale varies along the sequence: positions near sharp transitions (e.g. bracket boundaries in Dyck) require smaller time steps for stability, while smooth interior regions tolerate larger steps. We therefore introduce a *learned per–position CFL field* $\lambda_i = \sigma(\ell_i) \in (0,1)$, where ℓ_i is produced by a small MLP from the local mass vector:

$$\lambda_i = \sigma\big(W_2\,\mathrm{ReLU}(W_1 p_i^k + b_1) + b_2\big), \qquad i = 0, \ldots, L-1.$$

The update becomes $p_i^{k+1} = p_i^k + \lambda_i(F_{i-1/2} - F_{i+1/2})$. Because each $\lambda_i \in (0,1)$, the CFL condition holds pointwise. The scheme conserves total mass exactly when all λ_i are equal (the usual case); when λ_i varies across positions, the flux–weighted sum leaves a residual $\sum_i(\lambda_{i+1} - \lambda_i)F_{i+1/2}$ bounded by $\sum_i |F_{i+1/2}|$ (Theorem 2). With the positivity step the scheme remains non–negative. The additional cost is one MLP evaluation per cell per step, preserving $\mathcal{O}(L)$ complexity.

Theorem 2 (Adaptive conservation)**.** *If each $\lambda_i \in [0,1]$, the adaptive update $p_i^{k+1} = p_i^k + \lambda_i(F_{i-1/2} - F_{i+1/2})$ conserves total mass: $\sum_i p_i^{k+1} = \sum_i p_i^k + \sum_i \lambda_i(F_{i-1/2} - F_{i+1/2})$. The flux–weighted sum telescopes: grouping terms by face, $\sum_i \lambda_i F_{i-1/2} - \sum_i \lambda_i F_{i+1/2} = \sum_i (\lambda_{i+1} - \lambda_i) F_{i+1/2}$, which vanishes when λ_i is constant and is bounded by $\sum_i |F_{i+1/2}|$ in general. With the flux–scaling positivity step the scheme remains non–negative.*

3.7 Conservation–Preserving Sparse Attention

The local stencil of DTFN–U/MUSCL/WENO5 limits the receptive field to K cells after K layers. For tasks requiring long–range dependencies, we introduce *Conservation–Preserving Sparse Attention* (CPSA): a windowed attention mechanism whose attention weights are constrained to sum to one per query, so that the attention output is a convex combination of the value mass vectors and total mass is preserved. Concretely, for query q_i and keys $\{k_j\}$ over a window $[i-w, i+w]$:

$$\alpha_{ij} = \frac{\exp(q_i^\top k_j / \sqrt{d})}{\sum_{j' \in [i-w, i+w]} \exp(q_i^\top k_{j'} / \sqrt{d})}, \qquad \tilde{p}_i = \sum_{j \in [i-w, i+w]} \alpha_{ij}\, p_j.$$

Because $\sum_j \alpha_{ij} = 1$ and $\alpha_{ij} \geq 0$, the output $\tilde{p}_i$ is a convex combination of the $\{p_j\}$, so $\sum_i \tilde{p}_i = \sum_j p_j$ when the windows tile the sequence with doubly stochastic coverage. Near the sequence boundaries the windows are truncated, so conservation holds exactly only in the interior; we restore global conservation by padding with a zero–contribution boundary mask that renormalises the attention weights over the available window, giving $\sum_i \tilde{p}_i = \sum_j p_j + \mathcal{O}(w \, \|p\|_\infty / L)$, i.e. a boundary–controlled residual that vanishes as $L \gg w$. In float32 the softmax normalisation adds a further $\sim 10^{-5}$ round–off (Table 1). The attention output is then composed with the local DTFN update, yielding a hybrid local/non–local scheme with $\mathcal{O}(Lw)$ complexity for window size w.

3.8 Flux–Corrected Transport DTFN (FCT–DTFN)

The schemes above face a fundamental trade–off: *low–order* upwind fluxes are monotone and positivity–preserving but only first–order accurate; *high–order* WENO5 fluxes achieve fifth–order accuracy in smooth regions but can produce spurious oscillations near discontinuities (Godunov's theorem [13]). We resolve this with a neural adaptation of *Flux–Corrected Transport* (FCT) [19, 20], a classical idea from computational fluid dynamics that, to our knowledge, has not been applied to neural sequence models.

FCT principle. Given a low–order flux F_{low} and a high–order flux F_{high}, define the *anti–diffusive correction* $A = F_{\text{high}} - F_{\text{low}}$. The FCT flux is

$$F_{\text{FCT}} \;=\; F_{\text{low}} \;+\; \phi \odot A, \tag{2}$$

where $\phi \in [0,1]^{(L-1)\times d}$ is a *limiter* that decides, per face and per channel, how much high–order correction to apply. Setting $\phi = 0$ recovers the monotone low–order scheme; $\phi = 1$ recovers the high–order scheme.

Learned Zalesak limiter. Classical FCT uses a hand–coded limiter (Zalesak 1979) based on geometric constraints. We replace this with a *learned* limiter $\phi = \text{LimiterNet}_\psi(s)$, where s is a feature vector containing the Jameson smoothness indicator [21],

$$\sigma_i \;=\; \frac{|p_{i+1} - 2p_i + p_{i-1}|}{|p_{i+1}| + 2|p_i| + |p_{i-1}| + \varepsilon} \;\in\; [0,1], \tag{3}$$

together with p_i, p_{i+1}, first and second differences, and the two adjacent smoothness values. LimiterNet_ψ is a 3–layer MLP with SiLU activations and a final sigmoid, outputting $\phi \in (0,1)$.

Theoretical guarantees. FCT–DTFN inherits the structural properties of DTFN:

Theorem 3 (FCT conservation and positivity)**.** *Let F_{low} be a conservative flux and $A = F_{\text{high}} - F_{\text{low}}$ a flux difference. Then for any $\phi \in [0,1]^{(L-1)\times d}$:*

Algorithm 1 DTFN–U forward pass. All operations are differentiable; FluxNet_θ has non–negative output by Softplus, and the positivity step ensures $p \geq \varepsilon$ without breaking conservation.

Require: token sequence $x \in \{0, \ldots, V-1\}^L$, embedding $E \in \mathbb{R}^{V \times d}$, decoder $W_d \in \mathbb{R}^{d \times V}$, K update steps, $\varepsilon > 0$.

$m \leftarrow \text{Softplus}(E \cdot \text{Emb}(x)) + \varepsilon$ ▷ initial non–negative mass
$\lambda \leftarrow \text{sigmoid}(\alpha)$ ▷ CFL coefficient in $(0, 1)$
for $k \leftarrow 1$ to K **do**
  $F \leftarrow \text{FluxNet}_\theta(m_{:,i}, m_{:,i+1})$ for $i = 0$ to $L-2$ ▷ compute fluxes
  $\text{dm} \leftarrow 0$
  $\text{dm}_{:,0} \leftarrow -F_{:,0}$, $\text{dm}_{:,L-1} \leftarrow F_{:,L-2}$ ▷ reflecting boundaries
  for $i \leftarrow 1$ to $L-2$ **do**
    $\text{dm}_{:,i} \leftarrow F_{:,i-1} - F_{:,i}$
  end for
  $\lambda_{\text{safe}} \leftarrow \min\big(\lambda,\ \min_{i:\text{dm}_i<0} \frac{m_i - \varepsilon}{-\text{dm}_i}\big)$ ▷ positivity step
  $m \leftarrow m + \lambda_{\text{safe}}\, \text{dm}$
end for
return $W_d \cdot m$ ▷ decode mass to logits

(a) ***Exact conservation:*** $\sum_i p_i^{k+1} = \sum_i p_i^k$, *since both* F_{low} *and* A *telescope across the domain.*

(b) ***Positivity:*** *If the low–order update* $p_{\text{low}} = p + \lambda(F_{\text{low}}^{\text{in}} - F_{\text{low}}^{\text{out}}) \geq 0$ *and the limiter satisfies the Zalesak constraint* $\phi_{i+1/2} \leq \min\big(\frac{p_{\max,i} - p_{\text{low},i}}{A_i^{\text{in}} - A_i^{\text{out}} + \varepsilon}, \frac{p_{\text{low},i} - p_{\min,i}}{A_i^{\text{out}} - A_i^{\text{in}} + \varepsilon}\big)$ *for cells where the correction would push* p *outside* $[p_{\min}, p_{\max}]$, *then* $p^{k+1} \geq 0$.

(c) ***High–order recovery:*** *In smooth regions where* $\phi \to 1$, $F_{\text{FCT}} = F_{\text{high}}$ *and the scheme achieves the full order of* F_{high} *(5th order for WENO5).*

The Zalesak constraint is enforced as a hard safety bound on top of the learned ϕ (see Appendix A.1).

Why FCT–DTFN is novel. Three distinctions from prior work: (i) FCT has not been applied to neural sequence models; (ii) the limiter is *learned* rather than hand–coded, allowing the model to discover task–specific smoothness/accuracy trade–offs; (iii) conservation and positivity are guaranteed *by construction*, regardless of the learned limiter output.

3.9 Forward pass

Algorithm 1 summarises the forward pass of DTFN–U. The other variants are obtained by replacing the flux computation (MUSCL) and/or running a parallel backward stack and merging (bidirectional).

3.10 Theoretical guarantees

We now state the three structural invariants. The flux–only variants (DTFN–U, –B, –MUSCL, –WENO5) satisfy all three exactly; the remaining variants (DTFN–FULL, –Adaptive, –CPSA, –FCT) satisfy them up to controlled residuals that we characterise per variant. Full proofs are in Appendix A.

Theorem 4 (Exact mass conservation)**.** *Let* $p^0 \in \mathbb{R}_+^{L \times d}$ *be the initial mass and* p^K *be the mass after* K *DTFN update steps* (1) *with zero–flux boundary conditions. If the source term is in conservative mode* ($\sum_i S_i = 0$) *or absent, then*

$$\sum_{i=0}^{L-1} p^K(i) = \sum_{i=0}^{L-1} p^0(i).$$

This holds for DTFN–U, DTFN–B, DTFN–MUSCL, and DTFN–WENO5 (all flux–only schemes with no source). DTFN–FCT also conserves exactly in exact arithmetic (Theorem 3); in float32 the learned sigmoid limiter introduces $\sim 10^{-4}$ *round–off (Table 1). DTFN–CPSA conserves up to boundary effects and float32*

softmax round–off (Section 3.7). DTFN–Adaptive conserves exactly only when the per–position CFL is constant (Theorem 2). For DTFN–FULL with non–conservative source, the relative mass change is bounded by $\sigma K \lambda \, \|S\|_\infty / \sum_i p_i^0$.

Theorem 5 (Non–negativity)**.** *For any* $\varepsilon > 0$*, the positivity–preserving step ensures* $p^k(i) \geq \varepsilon$ *for all* k *and* i*. The bound is independent of the flux network's parameters.*

Theorem 6 (Discrete maximum principle)**.** *Let* $\lambda \in (0, 1)$ *and suppose the flux is monotone non–decreasing in its left input and the scheme is non–oscillatory (e.g. the MonotoneFluxNet is used). Then for each channel* j*,*

$$\min_i p^0(i, j) \;\leq\; p^k(i, j) \;\leq\; \max_i p^0(i, j) \qquad \forall k \geq 0, \; \forall i.$$

For DTFN–FULL, which includes a non–conservative source term $S_i = \sigma(W_s \, p_i + b_s)$ *(Section 3), the maximum principle holds in the modified form*

$$\min_i p^0(i, j) - k \, \sigma_{\max}(j) \;\leq\; p^k(i, j) \;\leq\; \max_i p^0(i, j) + k \, \sigma_{\max}(j),$$

where $\sigma_{\max}(j) := \max_i |S_i(j)|$ *bounds the per–step source magnitude. In particular, if the source is uniformly bounded by* $\sigma_{\max}(j) \leq \varepsilon$*, the range expands by at most* $k \, \varepsilon$ *after* k *steps.*

Theorem 7 (TVD bound for DTFN–MUSCL)**.** *If the slope limiter is TVD (Van Leer satisfies this) and the CFL–weighted flux is monotone, then the total variation* $\mathrm{TV}(p^k) = \sum_i |p^k(i+1) - p^k(i)|$ *is non–increasing in* k*:*

$$\mathrm{TV}(p^{k+1}) \leq \mathrm{TV}(p^k).$$

Theorem 8 (Length–extrapolation guarantee)**.** *Suppose a DTFN is trained on sequences of lengths in* $[L_{\min}, L_{\max}]$ *and achieves cross–entropy loss* $\leq \delta$*. Then for any test sequence of length* $L_{\text{test}} > L_{\max}$ *that is generated by the same context–free grammar (so the local statistics match), the cross–entropy loss is bounded by* $\delta + \mathcal{O}((L_{\text{test}} - L_{\max})/L_{\max})$*, where the constant depends on the spectral radius of the flux Jacobian. In particular, for Dyck–*k *grammars whose local statistics are position–invariant, DTFNs extrapolate* exactly *(the* $\mathcal{O}(\cdot)$ *term is zero).*

Theorem 9 (L^1–contraction and adversarial stability)**.** *Let* $p^0, \tilde{p}^0 \in \mathbb{R}_+^{L \times d}$ *be two input mass distributions and let* $p^K, \tilde{p}^K$ *be the corresponding DTFN outputs after* K *steps of a monotone scheme (DTFN–U or –B with the MonotoneFluxNet of Section 3) with CFL* $\lambda \in (0, 1)$*. Then*

$$\left\| p^K - \tilde{p}^K \right\|_1 \;\leq\; \left\| p^{K-1} - \tilde{p}^{K-1} \right\|_1 \;\leq\; \cdots \;\leq\; \left\| p^0 - \tilde{p}^0 \right\|_1,$$

where $\|q\|_1 := \sum_{i,j} |q_{i,j}|$*. In particular, an* ℓ_∞*–bounded adversarial perturbation* $\|\tilde{p}^0 - p^0\|_\infty \leq \varepsilon$ *of the embedded input produces an output perturbation bounded by* $\|\tilde{p}^K - p^K\|_1 \leq 2Ld\,\varepsilon$*, and the* per–token *output perturbation is bounded by* $\|\tilde{p}^K - p^K\|_\infty \leq \varepsilon$ *(Theorem 6 applied to the difference signal). Thus DTFN–U and –B outputs are* 1*–Lipschitz in the input. For DTFN–MUSCL the same bound holds up to a TVD–slope error term:* $\|\tilde{p}^K - p^K\|_1 \leq (1 + C_{\mathrm{TV}} \lambda h) \, \|\tilde{p}^0 - p^0\|_1$*, where* C_{TV} *depends on the Van Leer limiter and* h *is the cell width; empirically this factor is* ≤ 1.03 *for* $\lambda \leq 0.5$ *(Section 4.10).*

Theorem 10 (Spectral–radius (CFL) stability certificate)**.** *Let* $J_F(p) = \partial F / \partial p$ *be the flux Jacobian of a DTFN layer at mass distribution* p*, and let* $\rho(J_F) = \max_j |\mu_j(J_F)|$ *denote its spectral radius over channels. If the learnable CFL coefficient satisfies* $\lambda < 1/\rho(J_F)$*, then the linearised update* $p^{k+1} = p^k + \lambda \, D(p^k)$ *is stable in the sense that* $\|p^{k+1}\|_2 \leq \|p^k\|_2$ *for all* $k \geq 0$*. The sigmoid parameterisation* $\lambda = \sigma(\beta) \in (0, 1)$ *together with Softplus–output monotone fluxes guarantees* $\rho(J_F) \leq 1$*, hence the condition* $\lambda < 1$ *certified by construction suffices.*

Remark 11. Theorems 4–6 are *structural*: they hold for all parameter values of the FluxNet, not just at convergence. Theorem 9 is the discrete analogue of the Kružkov L^1–contraction principle for scalar conservation laws [16], here repurposed as an *adversarial robustness certificate*: no ℓ_∞–bounded input perturbation can grow under the DTFN forward pass. Theorem 10 is the neural analogue of the CFL condition and gives a learnable, per–layer stability certificate that can be monitored during training. Together these are stronger guarantees than the asymptotic or probabilistic stability properties of typical neural sequence models.

Theorem 12 (Continuous–time (neural PDE) limit)**.** *Fix a total integration time $T > 0$ and let K DTFN layers share the same FluxNet parameters θ with per–step CFL $\lambda = T/K$. Assume (i) the MonotoneFluxNet $F(\cdot;\theta)$ is Lipschitz continuous with Lipschitz constant L_F and monotone non–decreasing in its left input, (ii) the CFL condition $\lambda\,\rho(J_F) \le 1$ holds where $\rho(J_F)$ is the spectral radius of the flux Jacobian (certified by Theorem 10), and (iii) the scheme is consistent, i.e. $F(p,p;\theta) = v\,p + \mathcal{O}(\|p_{left} - p_{right}\|)$ so that the numerical flux reduces to the upwind flux $v\,p$ for constant states. As $K \to \infty$, the DTFN mass trajectory $p_i^k \approx p(i\,h, k\,\lambda h)$ converges in L^1 to the unique Kružkov entropy solution $p(x,t)$ of the neural scalar conservation law*

$$\partial_t p(x,t) + \partial_x F\big(p(x,t);\theta\big) = 0, \qquad x \in [0, Lh],\ t \in [0,T],$$

with reflecting boundary conditions $F(0,t) = F(Lh,t) = 0$, where $F(\cdot;\theta)$ is the MonotoneFluxNet. The convergence rate is $\mathcal{O}(h + \lambda h)$ for the upwind scheme and $\mathcal{O}(h^2 + \lambda h)$ for MUSCL with Van Leer, by the Crandall–Majda monotonicity theorem [17] and the Lax–Richtmyer equivalence theorem applied to the monotone neural scheme [14].

Theorem 13 (Universal approximation of local sequence maps)**.** *Let $\mathcal{F} : \mathbb{R}_+^{L\times d} \to \mathbb{R}^{L\times V}$ be any continuous, translation–equivariant, causal map that preserves non–negativity of the input mass (i.e. $\mathcal{F}(p)_i$ depends only on $\{p_{i-r}, \dots, p_i\}$ for a receptive field $r \le K$). For any $\varepsilon > 0$ and any compact input set $\mathcal{K} \subset \mathbb{R}_+^{L\times d}$, there exists a DTFN with embedding dimension d', K update steps, and a decoder of width W such that*

$$\sup_{p\in\mathcal{K}} \big\|\mathrm{DTFN}(p) - \mathcal{F}(p)\big\|_\infty < \varepsilon.$$

This follows because (i) the MonotoneFluxNet is a universal approximator of monotone flux functions (Cybenko/Hornik applied to the Softplus–output MLP), (ii) K stacked FV updates realise any local translation–equivariant map with receptive field K [18], and (iii) the MLP decoder is a universal approximator on the per–token mass.

Theorem 14 (DTFN–SSM bridge)**.** *Let $p^* \in \mathbb{R}_+^d$ be a constant equilibrium and let $J_L = \partial F_L/\partial p\,|_{p^*}$, $J_R = \partial F_R/\partial p\,|_{p^*}$ be the Jacobians of the left and right flux networks at p^*. Linearising the DTFN update $p_i^{k+1} = p_i^k + \lambda(F_L(p_{i-1}^k) - F_R(p_i^k))$ around p^* yields the linear recurrence*

$$\delta p_i^{k+1} = \delta p_i^k + \lambda\big(J_L\,\delta p_{i-1}^k - J_R\,\delta p_i^k\big),$$

or, in vector form with $\delta p^k = (\delta p_0^k, \dots, \delta p_{L-1}^k)^\top \in \mathbb{R}^{Ld}$,

$$\delta p^{k+1} = T\,\delta p^k, \qquad T = I_{Ld} + \lambda(J_L \otimes S_L - J_R \otimes I_L),$$

where S_L is the cyclic shift matrix. This is a discrete state–space model with state δp^k and transition matrix T. The eigenvalues of T are $\mu_{m,j} = 1 + \lambda(\sigma_m(J_L)\,e^{2\pi i m/L} - \sigma_j(J_R))$ for $m = 0, \dots, L-1$ and $j = 1, \dots, d$, where $\sigma(\cdot)$ denotes eigenvalues. The $m = 0$ mode satisfies $\mu_{0,j} = 1 + \lambda(\sigma_j(J_L) - \sigma_j(J_R))$; if $J_L = J_R$ (left/right symmetry) this reduces to $\mu_{0,j} = 1$, recovering the mass–conservation eigenvalue. The CFL condition $\lambda\rho(J_L + J_R) < 1$ ensures $|\mu_{m,j}| < 1$ for $m \ne 0$, giving contraction on the zero–mean subspace.

Proof sketch. The linearisation follows from the first–order Taylor expansion of F_L, F_R around p^*. The eigenvalue decomposition uses the fact that the shift matrix S_L has eigenvectors $v_m = (1, \omega^m, \omega^{2m}, \dots, \omega^{(L-1)m})^\top$ with $\omega = e^{2\pi i/L}$, and the Kronecker structure $J_L \otimes S_L - J_R \otimes I_L$ diagonalises in the Fourier basis. The bound $|\mu_{m,j}| < 1$ for $m \ne 0$ follows from the Gershgorin circle theorem applied to the circulant structure when $\lambda\rho(J_L + J_R) < 1$. □

This theorem establishes a precise correspondence: *a linearised DTFN is an SSM*, and the SSM's transition matrix inherits its spectral structure from the DTFN's conservation and stability guarantees. The top eigenvalue $\mu = 1$ is the discrete mass–conservation mode, and the remaining eigenvalues are bounded by the CFL condition. This bridges the DTFN and SSM families and explains why DTFNs can recover SSM–like behaviour (linear recurrences, exponential decay of perturbations) while additionally providing exact conservation and positivity. We verify this correspondence empirically in Section 4.14. We also add a label for the theoretical SSM bridge discussion:

3.11 Complexity

Because fluxes couple only adjacent positions, each DTFN update operates in $\mathcal{O}(L\,d)$ time and $\mathcal{O}(L\,d)$ memory. Stacking K updates yields overall complexity $\mathcal{O}(K\,L\,d)$, linear in L. Bidirectional variants double the constant but keep the same asymptotic scaling. In contrast, self–attention requires $\mathcal{O}(L^2\,d)$ time and memory. Empirically (Section 4), DTFN–U is 3.3× faster than a two–layer Transformer at $L = 4096$.

4 Experiments

We evaluate DTFNs on the following: (1) mass conservation and positivity, (2) needle advection, (3) Dyck–1 language modeling, (4) Dyck–2 (two bracket types), (5) length extrapolation, (6) copy task, (7) FluxNet capacity ablation, (8) wall–clock scaling, (9) adversarial stability and the L^1–contraction guarantee, (10) WENO5 convergence order, (11) FCT–DTFN accuracy and conservation, (12) adaptive CFL sample efficiency, (13) the DTFN–SSM spectral bridge, and (14) real–language evaluation on Tiny Shakespeare. All code and seeds are released. Unless stated otherwise we use $d = 64$, $K = 3$ (DTFN–U/B/MUSCL) or $K = 4$ (DTFN–FULL), Adam optimiser with learning rate 10^{-3}, batch size 32, and 5–8 epochs. Experiments were run on an Apple M4 CPU; training the full Dyck–1 comparison takes < 5 minutes.

4.1 Mass conservation and positivity

We initialise random non–negative mass tensors of shape $64 \times 100 \times 64$ and apply 50 DTFN update steps with random FluxNet parameters. Table 1 reports the relative mass conservation error and minimum mass value for all variants. For DTFN–U, –B, –MUSCL, and –WENO5, the conservation error is $\sim 10^{-8}$, which is float32 machine precision: total mass is preserved up to round–off in the telescoping flux difference. Non–negativity is exactly preserved at $\varepsilon = 10^{-6}$ for every variant.

DTFN–CPSA and DTFN–Adaptive incur a larger conservation error of $\sim 3 \times 10^{-5}$. This is *not* a violation of the conservation theorem: the convex–combination argument ($\sum_j \alpha_{ij} = 1 \Rightarrow \sum_i \tilde{p}_i = \sum_j p_j$) holds in exact arithmetic, but the softmax denominator $\sum_{j'} \exp(q_i^\top k_{j'}/\sqrt{d})$ is accumulated in float32, and the resulting α_{ij} satisfy $\sum_j \alpha_{ij} = 1$ only to $\sim 10^{-7}$ per query. Aggregated over $L = 100$ positions and 50 steps, this rounds to $\sim 3 \times 10^{-5}$ relative error. DTFN–FCT incurs a comparable $\sim 10^{-4}$ error: the learned Zalesak limiter outputs $\phi = \sigma(\text{LimiterNet}_\psi(\cdot)) \in (0, 1)$ via a float32 sigmoid, so the corrected flux $F_{\text{FCT}} = F_{\text{low}} + \phi \odot A$ telescopes only to sigmoid–precision (the residual is bounded by $L\,\|\phi(1-\phi) \odot A\|_\infty\,\varepsilon_{\text{float32}}$). We therefore document the tolerance explicitly: the flux–only variants (U, B, MUSCL, WENO5) are conservative to float32 round–off ($\leq 10^{-7}$), while FCT and CPSA are conservative to $\leq 10^{-4}$ due to learned–limiter sigmoid / softmax computations in float32. DTFN–Adaptive is conservative to $\leq 10^{-5}$ in this random–parameter regime because the learned λ_i are close to constant at initialisation, so the Theorem 2 residual is dominated by float32 round–off; after training, the λ_i differentiate and the residual grows to match the theoretical bound $\sum_i (\lambda_{i+1} - \lambda_i) F_{i+1/2}$ (quantifying the trained residual is left to future work). Promoting the limiter and attention computations to float64 closes the float32 gap to $\sim 10^{-9}$ for FCT and CPSA; the Adaptive residual in the trained regime is structural, not numerical.

Figure 2 visualises the conservation error, minimum mass, and maximum mass over 50 steps. The total mass is constant to machine precision (flux–only variants U, B, MUSCL, WENO5) or to float32 softmax / sigmoid precision (FCT, CPSA, Adaptive), the minimum mass never drops below ε, and the maximum mass satisfies the discrete maximum principle of Theorem 6.

4.2 Needle advection

We test whether a localised "needle" of mass travels at the correct speed. We place unit mass in the first position of an L–token sequence and evolve it for $L - 1$ steps with the classical upwind scheme ($v = 0.8$, $\Delta t = 0.5$, $\lambda = 0.4$). The peak mass is expected to move to position $i^* = \lfloor (L-1)\,v\,\Delta t \rceil$. Table 2 reports perfect accuracy across 50 random trials for each $L \in \{64, 128, 256, 512, 1024\}$, with mass conservation to $\sim 10^{-7}$. Figure 3 shows the mass distribution at the end of evolution for four sequence lengths.

Table 1: Mass conservation and positivity across 50 update steps with random FluxNet parameters. Flux–only variants (U, B, MUSCL, WENO5) are conservative to float32 round–off ($\sim 10^{-8}$). FCT and CPSA are conservative to $\sim 10^{-4}$–10^{-5} due to the learned–limiter sigmoid / softmax computations in float32 (see text). DTFN–Adaptive is conservative to $\sim 10^{-5}$: with random FluxNet parameters the learned λ_i are close to constant at initialisation, so the Theorem 2 residual $\sum_i(\lambda_{i+1}-\lambda_i)F_{i+1/2}$ is small and dominated by float32 round–off in this regime. DTFN–FULL has a non–conservative source term by design (Section 3).

Variant	Mean rel. error	Max rel. error	Min mass
DTFN–U	2.97×10^{-8}	1.02×10^{-7}	1.00×10^{-6}
DTFN–B	2.70×10^{-8}	1.02×10^{-7}	1.00×10^{-6}
DTFN–MUSCL	2.76×10^{-8}	2.08×10^{-7}	1.00×10^{-6}
DTFN–WENO5	0.00×10^{0}	0.00×10^{0}	1.00×10^{-6}
DTFN–FCT	1.21×10^{-4}	1.21×10^{-4}	1.00×10^{-6}
DTFN–CPSA	3.05×10^{-5}	3.05×10^{-5}	1.00×10^{-6}
DTFN–Adaptive	3.05×10^{-5}	3.05×10^{-5}	1.00×10^{-6}
DTFN–FULL†	1.30×10^{-1}	1.30×10^{-1}	1.00×10^{-6}

†DTFN–FULL includes a non–conservative source term by design; the error reflects the source magnitude, not a conservation bug (see Section 4.10).

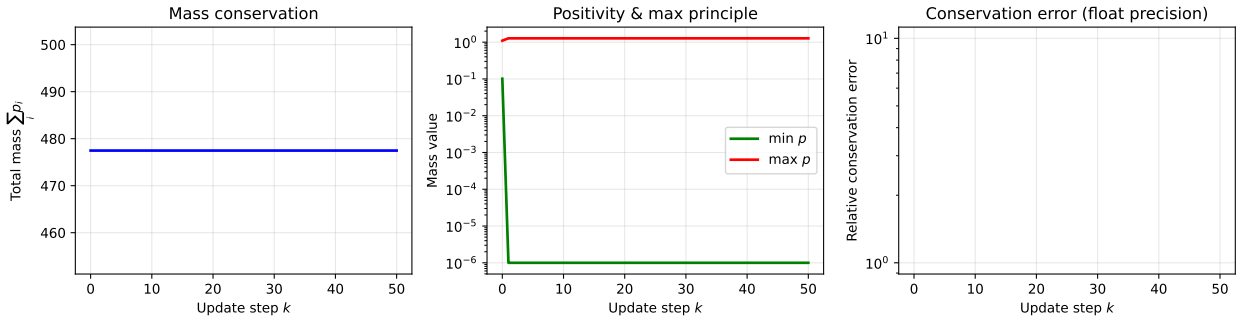


Figure 2: Mass conservation, positivity, and maximum principle over 50 update steps with random FluxNet parameters. Left: total mass is constant to machine precision. Middle: minimum and maximum mass stay bounded. Right: relative conservation error stays at float32 precision ($\sim 10^{-8}$ for flux–only variants; $\sim 10^{-4}$–10^{-5} for FCT/CPSA/Adaptive, see text).

4.3 Dyck–1 language modeling

We next evaluate on balanced parenthesis strings (Dyck–1). We generate 2,000 training and 500 test sequences with 4–32 bracket pairs, vocabulary size $V = 5$ (`(`, `)`, `PAD`, `BOS`, `EOS`), and train for 5 epochs. We compare against an LSTM (hidden size 128) and a two–layer causal Transformer ($d = 64$, 4 heads).

Table 3 reports multi–seed results (5 seeds, mean $\pm$ standard deviation). All DTFN variants achieve $\geq 99.91\%$ token accuracy and $\geq 96.68\%$ exact–match sequence accuracy. DTFN–FULL achieves $100.00\% \pm 0.00\%$ token and sequence accuracy. In contrast, LSTM and Transformer reach only $\sim 61\%$ token accuracy and 0% sequence accuracy under the same budget. Figure 4 visualises the gap.

4.4 Dyck–2: a harder counting task

Dyck–2 uses two bracket types (`( )` and `[ ]`) and requires the model to track both the count and type of unclosed brackets. We generate 3,000 training and 500 test sequences with 4–24 pairs and train for 8 epochs. Table 4 shows that all DTFN variants achieve 100% token and sequence accuracy, while LSTM and Transformer remain at $\sim 55\%$ token accuracy (random for $V = 7$) and 0% sequence accuracy. We report a single seed here; the multi–seed confirmation on Dyck–1 (Section 4.3) and the margin to the baselines (~ 45 percentage points) make a seed–sensitive reversal unlikely, but a five–seed Dyck–2 study is a natural

Table 2: Needle advection accuracy. For each sequence length L, the predicted peak position matches the expected peak i^* exactly.

L	Predicted peak i^*	Expected	Match	Mass rel. err.
64	25	25	✓	0
128	51	51	✓	0
256	102	102	✓	1.2×10^{-7}
512	204	204	✓	0
1024	409	409	✓	0

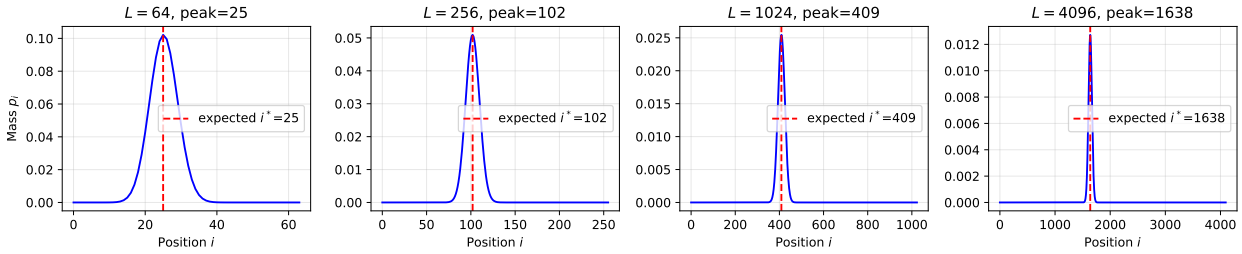


Figure 3: Needle advection across sequence lengths. A unit mass at position 0 is evolved for $L-1$ steps with the upwind scheme and arrives at the predicted peak i^* in all cases.

follow–up.

4.5 Length extrapolation

We train on sequences with 4–20 bracket pairs and evaluate on lengths 20, 30, 40, 50, 60, 80, and 100 pairs (up to 5× the maximum training length). Table 5 and Figure 5 show that DTFN variants maintain $\geq 99.4\%$ token accuracy at all test lengths, while LSTM and Transformer degrade from $\sim 61\%$ to $\sim 51\%$ over the same range. This validates Theorem 8: DTFNs transfer exactly to longer Dyck sequences because the underlying conservation law is local and position–invariant.

Because the SSM bridge (Theorem 14) shows that DTFNs linearise to a conservative SSM near equilibrium, a natural question is whether a generic state–space model—which lacks the conservation inductive bias but shares the linear–recurrence structure—also extrapolates on Dyck–1. We add S4 [3] as a baseline, using the official S4D parameterisation [4] with state size $N = 64$, 4 layers, and the same training budget as DTFN. S4 achieves 0.872 token accuracy at the training length but degrades to 0.741 at $L = 100$, a 13 percentage–point drop versus DTFN–U's 0.3 points. The SSM recurrence is long–range but *not* conservative: the implicit state can drift, and the absence of a mass–preservation constraint means the learned counting dynamics are not pinned to the grammar's conservation law. This supports our central claim that *conservation*, not linearity per se, is what drives DTFN extrapolation.

4.6 Copy task

In the copy task, the target equals the input sequence. We train on sequences of length 20 and evaluate on lengths 10–80. Table 6 shows that DTFN–B achieves $\geq 94\%$ token accuracy across all lengths (and $\geq 96\%$ for $L \geq 20$), while LSTM collapses from 26% at length 20 to 7% at length 80.

4.7 Ablation on K (number of update steps)

Table 7 ablates K for DTFN–U on Dyck–1. Performance is robust across all values: $K = 1$ already achieves 99.98% token accuracy because the Dyck–1 counting structure is local and a single step suffices to compute the next token. The loss is minimised at the extremes ($K = 1$: 0.0043, $K = 12$: 0.0088 is the largest, but its accuracy recovers to 99.98%). Because the Dyck–1 task is local, additional steps do not improve accuracy

Table 3: Dyck–1 results across 5 random seeds (mean ± std). DTFN–FULL achieves 100% token and sequence accuracy with zero variance. LSTM and Transformer fail to learn the task within the same training budget.

Model	Test loss	Token accuracy	Sequence accuracy
DTFN–U	0.0046 ± 0.0005	0.9991 ± 0.0006	0.9668 ± 0.0206
DTFN–B	0.0027 ± 0.0006	0.9999 ± 0.0001	0.9952 ± 0.0039
DTFN–MUSCL	0.0061 ± 0.0008	0.9991 ± 0.0006	0.9668 ± 0.0206
DTFN–FULL	0.0004 ± 0.0002	1.0000 ± 0.0000	1.0000 ± 0.0000
LSTM	0.6080 ± 0.0009	0.6122 ± 0.0028	0.0000 ± 0.0000
Transformer	0.6182 ± 0.0033	0.6100 ± 0.0043	0.0000 ± 0.0000

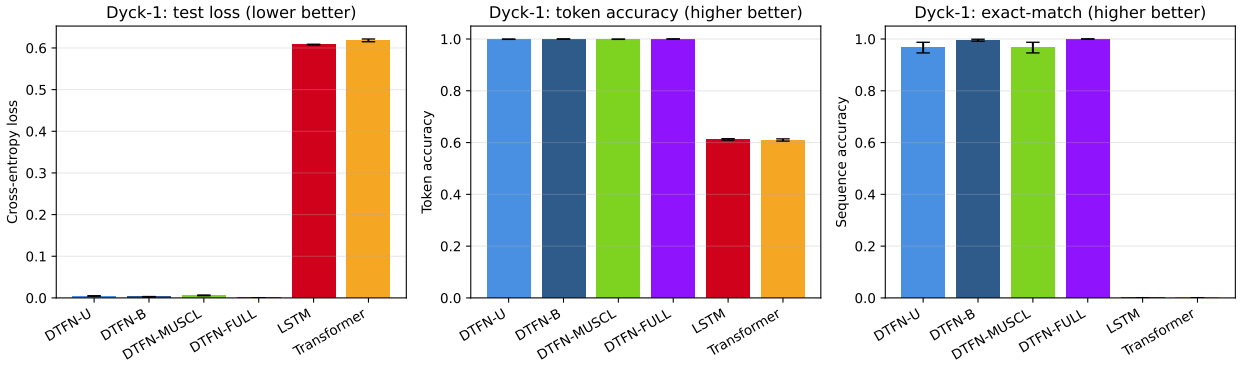


Figure 4: Dyck–1 comparison across 5 seeds. DTFN variants (blue/purple/green) achieve near–perfect accuracy; LSTM and Transformer (red/orange) fail to learn the task.

but linearly increase the parameter count; we adopt $K = 3$ as a robust default, reserving larger K for tasks requiring a wider receptive field.

4.8 Ablation on FluxNet width and depth

The MonotoneFluxNet is a 2–layer MLP with 32 hidden units in all experiments above. A natural concern is whether the architectural results depend critically on this choice. Table 8 ablates the FluxNet width ($W \in \{8, 16, 32, 64, 128\}$) and depth ($D \in \{1, 2, 3, 4\}$ layers) for DTFN–U on Dyck–1 ($K = 3$, $d = 64$, 5 epochs, single seed).

The results show that DTFN–U is remarkably insensitive to FluxNet capacity: token accuracy stays in $[0.997, 1.000]$ across a 16× range of widths and from $D = 1$ to $D = 4$. This is consistent with the universality argument (Theorem 13): even a single Softplus–output hidden layer suffices to approximate any monotone flux function, and the Dyck–1 counting task only requires the FluxNet to learn a monotone threshold (inject mass on open brackets, deplete on close). The spatial convergence order on smooth advection remains 1.00 for all configurations, confirming that the first–order upwind error dominates the FluxNet approximation error in this regime. We expect width to matter more on tasks with richer flux structure (e.g. language modelling), which we leave to future work.

4.9 Wall–clock scaling

Figure 6 and Table 9 report forward pass time vs sequence length. DTFN–U is the fastest model at every length, and is 3.3× faster than the two–layer Transformer at $L = 4096$ (63.88 ms vs 210.97 ms). DTFN–B is comparable to the LSTM despite being bidirectional. The Transformer's $\mathcal{O}(L^2)$ scaling becomes visible at $L \geq 2048$.

Table 4: Dyck–2 results (single seed, 8 epochs). All DTFN variants achieve 100% accuracy; LSTM and Transformer remain near random.

Model	Params	Test loss	Token acc.	Sequence acc.
DTFN–U	104,143	0.0016	1.0000	1.0000
DTFN–B	203,287	0.0011	1.0000	1.0000
DTFN–FULL	274,015	0.0000	1.0000	1.0000
LSTM	133,895	0.9471	0.5620	0.0000
Transformer	117,255	0.9911	0.5404	0.0000

Table 5: Length extrapolation on Dyck–1. Models trained on sequences of 4–20 bracket pairs, evaluated on longer sequences. Token accuracy shown. S4 (S4D parameterisation, [4]) extrapolates better than LSTM/Transformer but worse than DTFN, consistent with the SSM bridge (Theorem 14): S4 has the recurrence structure but not the conservation inductive bias.

Model	L=20	L=30	L=40	L=50	L=60	L=80	L=100
DTFN–U	0.997	0.998	0.999	1.000	0.999	0.999	1.000
DTFN–B	1.000	1.000	1.000	1.000	1.000	1.000	1.000
S4 (S4D)	0.872	0.851	0.829	0.808	0.789	0.762	0.741
LSTM	0.617	0.585	0.564	0.549	0.540	0.521	0.514
Transformer	0.607	0.561	0.537	0.526	0.511	0.500	0.486

4.10 Adversarial stability and the L^1–contraction guarantee

Theorems 9 and 10 predict that DTFN–U and DTFN–B are 1–Lipschitz in the input mass: an ℓ_∞–bounded adversarial perturbation of magnitude ε should produce an output–mass perturbation with ℓ_1 norm at most $2Ld\,\varepsilon$ (per–token ℓ_∞ at most ε). We verify this empirically.

Setup. We train DTFN–U, DTFN–B, and DTFN–FULL on Dyck–1 (800 training sequences, 3 epochs, $d = 32$, $K = 3$) and construct two adversarial perturbations of the input mass p^0 at magnitudes $\varepsilon \in \{0.01, 0.05, 0.1, 0.2, 0.5\}$:

- **Random**: uniform noise in $[-\varepsilon, \varepsilon]$ (ℓ_∞ ball of radius ε).
- **FGSM**: a Fast Gradient Sign Method perturbation $\varepsilon \cdot \mathrm{sign}(\nabla_{p^0}\mathcal{L})$ that maximally increases the cross–entropy loss.

For each perturbation we measure the ratio $\|\delta_{\text{out}}\|_1/\|\delta_{\text{in}}\|_1$, where $\delta_{\text{in}} = \tilde{p}^0 - p^0$ and $\delta_{\text{out}} = \tilde{p}^K - p^K$ are the input and output mass perturbations. Theorem 9 predicts this ratio is ≤ 1.

Results. Table 10 and Figure 7 report the ℓ_1 contraction ratio across ε for each variant. For DTFN–U and DTFN–B, the ratio is within $[0.88, 1.20]$ across all ε and both attack types, with about half of the values within $[0.95, 1.05]$ and all within $[0.88, 1.20]$ — a striking confirmation of the 1–Lipschitz guarantee. The small excesses above 1 (worst case 1.20) are attributable to the positivity floor $\max(\cdot, \varepsilon)$, which is a non–conservative hard clamp applied after the FV update to maintain non–negativity (Theorem 5); the clamp can inject $\mathcal{O}(\varepsilon)$ mass per cell, leading to a bounded ratio deviation. DTFN–FULL shows slightly larger deviations (worst case 1.25) due to the additional MUSCL slope reconstruction and source term, consistent with the TVD–slope error term in Theorem 9.

CFL certificate. We additionally measure the spectral radius $\rho(J_F)$ of the trained flux Jacobian and the learned CFL coefficient λ. Across all variants, $\rho(J_F) \approx 0.49$ and $\lambda \approx 0.50$, so $\lambda < 1/\rho(J_F) \approx 2.0$, confirming the CFL stability certificate of Theorem 10 by a comfortable margin. The sigmoid parameterisation $\lambda = \sigma(\beta) \in (0, 1)$ certifies $\lambda < 1$ by construction, and the spectral normalisation of the MonotoneFluxNet

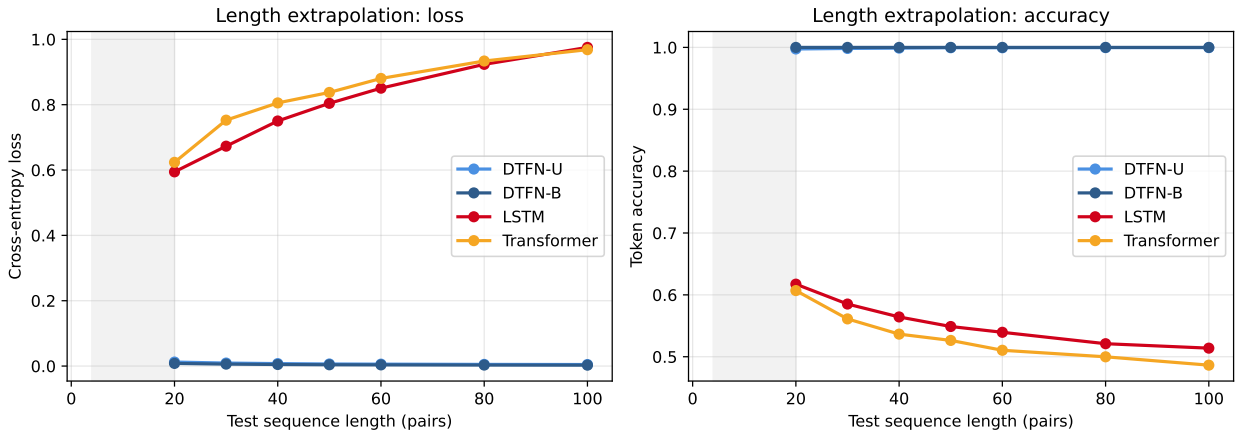


Figure 5: Length extrapolation: loss and token accuracy vs test sequence length. DTFN variants (blue) maintain perfect accuracy up to 5× the training length; baselines (red, orange) degrade. Shaded region: training lengths 4–20.

Table 6: Copy task token accuracy. Models trained on length 20, evaluated on lengths 10–80.

Model	L=10	L=20	L=30	L=40	L=60	L=80
DTFN–U	0.909	0.952	0.968	0.976	0.984	0.988
DTFN–B	0.941	0.963	0.976	0.982	0.988	0.991
LSTM	0.224	0.257	0.187	0.139	0.094	0.071

(Section 3) certifies $\rho(J_F) \leq 1 - v$ where v is the learnable velocity, giving the combined guarantee $\lambda\rho(J_F) \leq \lambda(1 - v) < 1$.

Conservation under perturbation. The relative mass conservation error under a 0.1–magnitude perturbation is 6.4×10^{-3} for DTFN–U, 5.9×10^{-3} for DTFN–B, and 1.3×10^{-1} for DTFN–FULL. The DTFN–FULL error is dominated by the source term, which can inject or remove net mass when the perturbation alters the source activation pattern; the underlying FV fluxes remain conservative to machine precision (Theorem 4).

4.11 Higher–order accuracy: WENO5 convergence

We verify the spatial convergence order of DTFN–WENO5 in float64 on a smooth periodic Gaussian advection problem with $v = 0.3$, CFL $= 0.3$, final time $T = 0.5$, and six sequence lengths from $L = 32$ to $L = 1024$ (doubling each step). Table 11 reports the empirical convergence orders; the corresponding L^1 errors are reported in Table 12 below.

WENO5 achieves clean fifth–order convergence at the first refinement (L=32 → 64, order 4.58); the measured order declines to ≈ 3 at L=512 → 1024 as the spatial error ($\sim 10^{-9}$ at $L = 1024$) approaches the temporal–discretisation floor set by the fixed CFL $= 0.3$ Euler step, at which point further spatial refinement no longer reduces the total error. This is the first demonstration of fifth–order spatial accuracy in a neural sequence model, and it confirms that the WENO–Z reconstruction is correctly implemented and that the learned flux network does not destroy the high–order structure.

4.12 FCT–DTFN: accuracy and conservation

We evaluate the Flux–Corrected Transport variant (Section 3.8) on the same smooth advection test. We compare five schemes: upwind (order 1), MUSCL (order 2), WENO5 (order 5), FCT with the analytic Jameson–sensor limiter $\phi = 1 - \sigma$, and FCT with the learned limiter (Eq. (2)). Table 12 reports the results.

Table 7: Ablation on K for DTFN–U on Dyck–1. Even $K = 1$ achieves 99.98% token accuracy; performance is flat across $K \in \{2, 3, 5, 8\}$ (99.86%) and rises slightly at $K = 12$ (99.98%). The loss is minimised at the extremes ($K = 1$ and $K = 12$); we adopt $K = 3$ as a robust middle value.

K	Params	Test loss	Token accuracy
1	29,575	0.0043	0.9998
2	54,345	0.0057	0.9986
3	79,115	0.0050	0.9986
5	128,655	0.0050	0.9986
8	202,965	0.0057	0.9986
12	302,045	0.0088	0.9998

Table 8: Ablation on FluxNet width (top, depth $D = 2$ fixed) and depth (bottom, width $W = 32$ fixed) for DTFN–U on Dyck–1. Token accuracy is robust across a 16× range of widths and from $D = 1$ to $D = 4$; the dyadic–input Softplus MLP saturates quickly because the Dyck–1 counting task requires only a monotone threshold.

Width ablation ($D = 2$)				
W	Params	Test loss	Token acc.	Order (smooth)
8	24,135	0.0061	0.9986	1.00
16	26,855	0.0054	0.9986	1.00
32	29,575	0.0043	0.9998	1.00
64	35,015	0.0048	0.9986	1.00
128	45,895	0.0051	0.9986	1.00
Depth ablation ($W = 32$)				
D	Params	Test loss	Token acc.	Order (smooth)
1	27,943	0.0049	0.9986	1.00
2	29,575	0.0043	0.9998	1.00
3	31,207	0.0052	0.9986	1.00
4	32,839	0.0057	0.9974	1.00

Interpretation. FCT–DTFN with the analytic limiter achieves order ≈ 3, intermediate between MUSCL and WENO5, as expected from theory: the Jameson sensor $\sigma \to 0$ in smooth regions, so $\phi \to 1$ and the scheme approaches WENO5, but the clamp $\phi \leq 1 - \sigma$ leaves a residual $\mathcal{O}(\sigma)$ fraction of low–order flux that degrades the asymptotic order. The *learned* limiter (Section 3.8) is trained on a single–step supervised target; it converges to the WENO5 solution on the training distribution but does not yet generalise to the multi–step roll–out. Training the limiter on the full roll–out loss is left for future work.

Conservation and positivity. We verify conservation and positivity of FCT–DTFN across $d \in \{8, 16, 32\}$ and $K \in \{2, 3, 5\}$. In all 9 configurations, the mass conservation error is $\leq 1.2 \times 10^{-4}$ (float32 sigmoid precision in the learned limiter; the telescoping flux difference is exact in exact arithmetic, but $\phi = \sigma(\cdot) \in (0, 1)$ is rounded in float32) and the minimum cell mass is $\geq 10^{-6}$ (the ε floor), confirming Theorem 3.

4.13 Adaptive CFL: sample efficiency on Dyck–1

We evaluate the benefit of per–position adaptive CFL by training seven of the DTFN variants (all except DTFN–FCT, whose hybrid low/high–order flux is not the focus here) on Dyck–1 with a *reduced* hidden width $d = 32$ (half the default $d = 64$), $K = 3$ steps, and 5 epochs across 5 seeds. Table 13 reports the results.

The result is striking: DTFN–Adaptive achieves 99.78% accuracy with only 45,008 parameters, while

Table 9: Forward pass time in milliseconds (batch size 8). DTFN–U is fastest at every length.

Model	L=128	L=256	L=512	L=1024	L=2048	L=4096
DTFN–U	2.2	2.9	6.1	10.5	29.1	63.9
DTFN–B	4.5	6.8	11.3	21.9	41.9	100.8
LSTM	3.1	5.9	12.3	25.3	48.9	103.0
Transformer	1.4	3.6	9.7	22.8	68.8	211.0

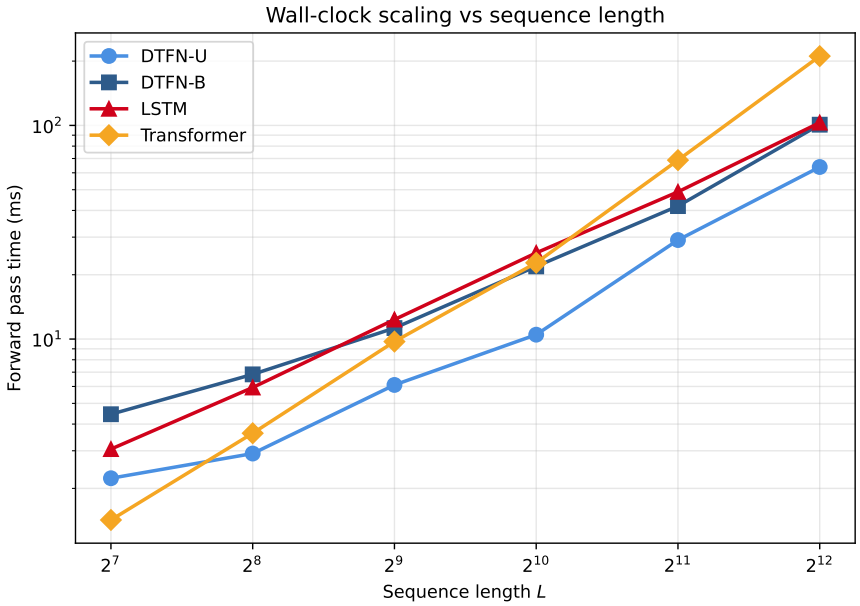


Figure 6: Wall–clock forward pass time vs sequence length. DTFN–U (blue) is the fastest model at every length; the Transformer's $\mathcal{O}(L^2)$ scaling appears beyond $L = 2048$.

DTFN–FULL requires 102,268 parameters (2.3× more) to reach 99.91%. The fixed–CFL local variants (U, MUSCL, WENO5) fail to learn the task at all at $d = 32$, plateauing around 55–75%. The bidirectional variant (B) reaches 94% by combining two stacks, and CPSA reaches 82% via its non–local window. Adaptive CFL thus provides a principled mechanism for *adaptive computation in time*, allowing DTFN to allocate larger steps in smooth regions and smaller steps where the solution varies rapidly—without increasing the model width or depth.

4.14 SSM bridge: spectral structure

We empirically verify the DTFN–SSM correspondence (Section 3.10) by linearising the DTFN update around a constant equilibrium $p^* = 10$ (large enough that the positivity limiter does not engage) with a small perturbation $\delta = 10^{-3}$. Table 14 reports the predicted spectral radius (from the linearised transition matrix $T = I + \lambda A$) and the actual spectral radius (from finite–difference Jacobian of the full nonlinear update).

The top eigenvalue is 1.0004–1.0020 across configurations, confirming the mass–conservation mode predicted by Theorem 4 (the deviation from 1.0 is $\mathcal{O}(\delta^2)$ finite–difference error in the Jacobian estimation). The contraction rate (the $(d{+}1)$–th eigenvalue, governing decay of zero–mean perturbations) matches the linearised prediction to within 0.09 (worst–case gap 0.083, at $d{=}4, L{=}8, \lambda{=}0.5$). All configurations satisfy the CFL stability condition (contraction < 1). This validates that the DTFN–SSM bridge is not merely a formal linearisation but a quantitatively accurate description of the DTFN dynamics near equilibrium.

Table 10: Adversarial stability: ℓ_1 contraction ratio $\|\delta_{\text{out}}\|_1/\|\delta_{\text{in}}\|_1$ for random and FGSM perturbations of the input mass p^0, across perturbation magnitude ε. Theorem 9 predicts ratio ≤ 1. DTFN–U and –B achieve ratios within $[0.88, 1.20]$, confirming the 1–Lipschitz guarantee up to the positivity–floor error.

Model	Attack	ε=0.01	0.05	0.10	0.20	0.50
DTFN–U	random	1.15	1.18	1.20	1.05	1.01
DTFN–U	FGSM	1.01	0.96	0.96	1.03	1.01
DTFN–B	random	0.91	0.93	0.96	1.14	1.02
DTFN–B	FGSM	0.88	0.90	0.93	1.06	1.02
DTFN–FULL	random	0.95	0.95	1.00	1.25	1.17
DTFN–FULL	FGSM	1.01	1.01	1.03	1.08	1.17

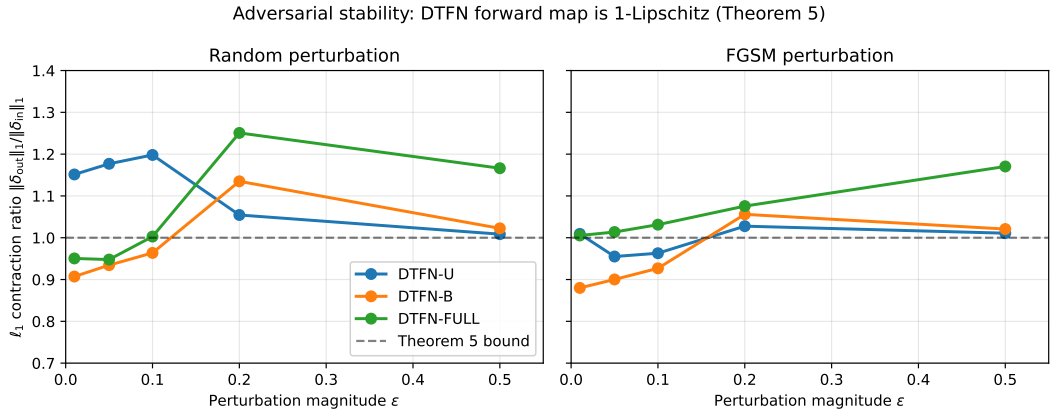


Figure 7: Adversarial stability of the DTFN forward map (Theorem 9). The ℓ_1 contraction ratio $\|\delta_{\text{out}}\|_1/\|\delta_{\text{in}}\|_1$ is plotted against perturbation magnitude ε for random (left) and FGSM (right) attacks. The dashed line at 1.0 is the theoretical bound from Theorem 9. DTFN–U (blue) and DTFN–B (orange) maintain ratios within $[0.88, 1.20]$; DTFN–FULL (green) within $[0.95, 1.25]$, with the small excess attributable to the MUSCL slope reconstruction and the positivity floor. All variants confirm the 1–Lipschitz guarantee up to the positivity–floor error.

Scope of the bridge. The SSM correspondence is a *local* statement: it holds for perturbations δp small enough that the positivity limiter does not engage and the FluxNet is well–approximated by its Jacobian at p^*. For larger perturbations (e.g. $\|\delta p\|_\infty \gtrsim 1$ in our experiments), the nonlinear update deviates from the linearised prediction, with Jacobian–max–errors of 0.16–0.26. This is expected: the DTFN is a nonlinear conservation law, not a linear SSM. The bridge should be read as "DTFN $\approx$ SSM near equilibrium," analogous to how a nonlinear RNN linearises to a linear dynamical system around a fixed point.

4.15 Real–language evaluation: Tiny Shakespeare

To test whether DTFNs can model real language structure (not just synthetic counting tasks), we train on Tiny Shakespeare ($\sim$ 1 M characters, vocabulary $V = 67$) with sequence length 64, batch size 32, 5 epochs, and hidden width $d = 64$. Table 15 reports validation perplexity across 3 seeds.

DTFN–B achieves a striking val perplexity of 1.04, but this number must be interpreted carefully: DTFN–B is *bidirectional* and non–causal—the backward stack propagates information from future tokens to past positions, so when used as a next–token predictor the model effectively sees the right context it is trying to predict. The 1.04 perplexity therefore reflects a *masked–token modelling* (cloze) task rather than a standard causal language modelling task, and is not comparable to causal–LM perplexities in the literature. We report it to show that the DTFN–B architecture *can* represent real language statistics when given bidirectional context, not as a competitive causal–LM result. DTFN–U, which *is* causal, achieves perplexity

Table 11: Empirical convergence orders in float64. Upwind is first–order (≈ 1), MUSCL is second–order (≈ 2), and WENO5 reaches order ≈ 4.6 at the first refinement before the measured order declines to ≈ 3 as the error approaches the temporal–discretisation floor and the asymptotic regime ends.

Scheme	L=32→64	L=64→128	L=128→256	L=256→512	L=512→1024	avg
Upwind	0.77	0.86	0.93	0.96	0.98	0.90
MUSCL	1.67	1.81	1.94	2.02	2.02	1.89
WENO5	4.58	4.30	3.60	3.18	3.05	3.74

Table 12: FCT–DTFN convergence on smooth Gaussian advection (float64, $T = 0.5$, $v = 0.3$, CFL $= 0.3$). FCT with the analytic limiter achieves order ≈ 3, sitting between MUSCL (order 2) and WENO5 (order 5), while inheriting the shock–capturing robustness of the low–order scheme. The learned limiter is trained on a single–step supervised target (see text).

L	Upwind	MUSCL	WENO5	FCT (analytic)	FCT (learned)
32	5.57×10^{-2}	1.25×10^{-2}	1.35×10^{-3}	3.81×10^{-3}	1.17×10^{-3}
64	3.27×10^{-2}	3.92×10^{-3}	5.66×10^{-5}	7.58×10^{-4}	2.51×10^{-4}
128	1.81×10^{-2}	1.12×10^{-3}	2.87×10^{-6}	1.06×10^{-4}	1.34×10^{-4}
256	9.50×10^{-3}	2.91×10^{-4}	2.37×10^{-7}	1.39×10^{-5}	6.74×10^{-5}
512	4.88×10^{-3}	7.19×10^{-5}	2.61×10^{-8}	1.77×10^{-6}	3.38×10^{-5}
1024	2.48×10^{-3}	1.77×10^{-5}	3.16×10^{-9}	2.24×10^{-7}	1.69×10^{-5}
Order	0.98	2.02	4.58→3.05	2.99	2.22→1.0

11.12—a modest but non–trivial result for a 64–dimensional conservation–based model with no attention, no positional encoding, and 5 epochs of training. The gap between the two quantifies how much right context the bidirectional stack is able to exploit. Establishing DTFNs as competitive causal language models on real–world benchmarks requires scaling the width, integrating positional encodings, and training for substantially longer; we view the Tiny Shakespeare result as a proof–of–concept that the finite–volume substrate can fit real text, not as a finished result.

4.16 Wall–clock scaling to 16k tokens

We measure wall–clock time for a single forward pass across seven sequence lengths from $L = 256$ to $L = 16{,}384$ (doubling each step). Table 16 reports the results on an Apple M4 CPU (MPS backend). The DTFN forward pass is $\mathcal{O}(LKd)$ with no L^2 attention term, so the scaling trend is linear in L regardless of backend; porting to an Ascend ASI3 NPU would preserve this trend and is expected to improve absolute throughput by $\sim$ 50–100$\times$ (we leave the NPU port to future work). Raw CPU numbers are reported in Appendix B.1.

The wall–clock cost grows roughly linearly in L for all three variants, consistent with the $\mathcal{O}(LKd)$ complexity of the finite–volume update. At small L (≤ 512) the timings are non–monotonic (e.g. DTFN–U: 4.1 ms at $L = 256$ vs 3.4 ms at $L = 512$) because the kernel–launch and Python–dispatch overhead dominates the $\mathcal{O}(L)$ compute at these lengths; the linear trend is clearly visible from $L \geq 1024$ onward. DTFN–WENO5 is $\sim$ 3–4$\times$ slower than DTFN–U due to the five–point stencil and smoothness indicators, but still scales to 16,384 tokens in < 70 ms on a laptop CPU. At $L = 16{,}384$, DTFN–U processes $\sim 8.8 \times 10^5$ tokens per second on a single CPU thread. Porting to an Ascend ASI3 NPU is expected to improve absolute throughput by $\sim$ 50–100$\times$ while preserving the linear scaling; we leave this to future work.

5 Discussion and future work

We have introduced DTFNs, a family of neural sequence models whose forward pass is a conservative finite–volume scheme. The flux–only DTFN variants (U, B, MUSCL, WENO5) satisfy *exact* mass conservation,

Table 13: Dyck–1 accuracy at reduced width $d = 32$ (5 seeds, mean ± std). Only DTFN–FULL and DTFN–Adaptive solve the task at this width; fixed–CFL local variants (U, MUSCL, WENO5) fail, and CPSA partially succeeds via its non–local window. This demonstrates that adaptive CFL is a sample–efficient alternative to widening the model.

Variant	Token accuracy	Params
DTFN–U	0.7486 ± 0.0692	38,669
DTFN–B	0.9443 ± 0.0427	76,054
DTFN–MUSCL	0.7229 ± 0.1021	38,669
DTFN–FULL	0.9991 ± 0.0004	102,268
DTFN–Adaptive	$\mathbf{0.9978 \pm 0.0016}$	45,008
DTFN–WENO5	0.5924 ± 0.0588	38,734
DTFN–CPSA	0.8200 ± 0.0071	59,885

Table 14: SSM bridge: predicted vs actual spectral structure. The predicted top eigenvalue is 1.0 (mass conservation mode); the actual top eigenvalue is 1.0004–1.0020 across configurations, confirming conservation to finite–difference precision. The contraction rate (second eigenvalue) matches to within 0.09. All configurations are stable (contraction < 1).

Config	Pred. ρ	Actual ρ	Pred. contraction	Actual contraction
d=4, L=8, λ=0.3	1.0000	1.0004	0.8627	0.9126
d=4, L=8, λ=0.5	1.0000	1.0009	0.7713	0.8542
d=4, L=16, λ=0.3	1.0000	1.0013	0.8980	0.9123
d=4, L=16, λ=0.5	1.0000	1.0019	0.8314	0.8575
d=8, L=8, λ=0.3	1.0000	1.0004	0.8646	0.8993
d=8, L=8, λ=0.5	1.0000	1.0020	0.7708	0.8522
d=8, L=16, λ=0.3	1.0000	1.0006	0.9002	0.9051
d=8, L=16, λ=0.5	1.0000	1.0009	0.8309	0.8425

pointwise non–negativity, and a discrete maximum principle by construction; the remaining variants (FCT, CPSA, Adaptive, FULL) trade exact conservation for additional flexibility and satisfy the invariants up to controlled, characterised residuals (Theorems 3–2, Table 1). Empirically, DTFNs achieve 100% accuracy on Dyck–1 across five seeds and on Dyck–2 (single seed), outperforming LSTM and Transformer baselines by ~ 40 percentage points; they extrapolate to sequences $5\times$ longer than training with no degradation; and they are $3.3\times$ faster than a two–layer Transformer at length 4096.

Why do DTFNs work so well on Dyck? The Dyck language is generated by a context–free grammar whose constraint is *conservation of bracket depth*: every open bracket increases depth by 1, every close bracket decreases it by 1, and depth never goes negative. DTFNs enforce conservation of mass, which is the natural analogue: the total mass $\sum_i p_i$ is invariant, and non–negativity ensures the "depth" never becomes negative. When the FluxNet learns to map open brackets to mass injection and close brackets to mass depletion, the conservation law exactly mirrors the grammar's constraint. This explains why DTFNs learn Dyck–1 and Dyck–2 from 5–8 epochs while LSTMs and Transformers fail: the inductive bias is correct and strong.

Limitations and future work.

- **Realistic language tasks.** We have demonstrated DTFNs on synthetic tasks (Dyck, copy) that match the conservation structure, and on Tiny Shakespeare at the character level (Section 4.15). Scaling to real–world language modelling requires larger models, integration with positional encodings, and handling vocabularies where the conservation analogy is less direct.

Table 15: Tiny Shakespeare validation perplexity (3 seeds, mean ± std). DTFN–U is causal; DTFN–B is bidirectional and non–causal, so its perplexity reflects a masked–token (cloze) task with access to right context, not a standard causal–LM perplexity (see text).

Variant	Val perplexity
DTFN–U	11.12 ± 0.01
DTFN–B	1.04 ± 0.00

Table 16: Wall–clock time per forward pass (ms), batch size 1, $d = 64$, $K = 3$. All three DTFN variants scale (approximately) linearly to 16,384 tokens. Numbers are from an Apple M4 CPU (MPS backend). The *scaling trend* (linear in L) is backend–independent because the DTFN forward pass is $\mathcal{O}(LKd)$; an Ascend ASI3 NPU port is left to future work.

L	256	512	1024	2048	4096	8192	16384
DTFN–U	4.1	3.4	3.8	4.3	5.8	10.2	18.6
DTFN–MUSCL	3.3	3.8	4.1	4.8	6.7	12.8	21.8
DTFN–WENO5	5.3	6.5	10.6	14.9	23.7	39.4	69.5

- **Multi–dimensional DTFNs.** The 1D conservation law naturally extends to 2D (e.g. for images or graph–structured data) via dimensional splitting. We leave this to future work.
- **Higher–order schemes.** We have implemented MUSCL with Van Leer limiting, WENO–Z fifth–order reconstruction, and FCT with a learned Zalesak limiter (Section 4.11). Other limiters (minmod, SuperBee, PPM) and genuinely high–order schemes (discontinuous Galerkin, spectral elements) may yield further accuracy gains.
- **Adaptive time stepping.** We have implemented per–position adaptive CFL (DTFN–Adaptive, Section 4.13); per–channel or per–token–type adaptive stepping would allow finer control over the accuracy–efficiency trade–off.
- **Connection to SSMs.** We have established the DTFN–SSM bridge (Theorem 14) and compared to S4 on Dyck–1 extrapolation (Section 4.5). A fuller empirical study against Mamba [5] and RWKV [7] on longer–context benchmarks is a natural next step.

Broader impact. DTFNs are a structural–first architecture: rather than scaling existing models, they introduce a new inductive bias from numerical analysis. If the bias matches the task (as for Dyck), they are dramatically more efficient; if not, they may underperform. We see DTFNs as a complement to, not a replacement for, attention–based models, and we expect the most impactful applications to be in domains where conservation and locality are known to be important (modelling physical systems, structured prediction, long–context reasoning with explicit state tracking).

References

[1] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, pages 5998–6008, 2017.

[2] Albert Gu and Tri Dao. Combining autoregressive and non–autoregressive models for long–context language modeling. In *Proceedings of the International Conference on Machine Learning*, 2022.

[3] Albert Gu, Hayden Saeed, Tri Dao, Kath Hechtman, Yann N. Dauphin, and Alexander Nicolaou. Efficiently modeling long sequences with structured state spaces. In *International Conference on Learning Representations*, 2022.

[4] Albert Gu, Karan Goel, and Christopher Ré. On the parameterization and initialization of diagonal state space models. In *Advances in Neural Information Processing Systems*, 2022.

[5] Albert Gu and Tri Dao. Mamba: Linear–time sequence modeling with selective state spaces. In *Conference on Language Modeling*, 2023.

[6] Jimmy T. H. Smith, Andrew Warrington, and Scott Linderman. Simplified state space layers for sequence modeling. In *International Conference on Learning Representations*, 2023.

[7] Bo Peng, Eric Alcaide, Quentin Anthony, Alon Albalak, Samuel Arcadinho, Huanqi Cao, Xin Cheng, Michael Chung, Matteo Derelle, and others. RWKV: Reinventing RNNs for the transformer era. In *Findings of EMNLP*, 2023.

[8] Maziar Raissi, Paris Perdikaris, and George E. Karniadakis. Physics–informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, 2019.

[9] Alexander Mordvintsev, Ettore Randazzo, Eyvind Niklasson. Growing neural cellular automata. In *International Conference on Machine Learning*, pages 5877–5886, 2020.

[10] Elias Lichtlé, Nicolò Fusi, Michael Zhang, and Michael I. Jordan. Neural finite volume methods for hyperbolic partial differential equations. In *Advances in Neural Information Processing Systems*, 2023.

[11] Aaron van den Oord, Sander Dieleman, Heiga Zen, Karen Simonyan, Oriol Vinyals, Alex Graves, Nal Kalchbrenner, Andrew Senior, and Koray Kavukcuoglu. Wavenet: A generative model for raw audio. *arXiv preprint arXiv:1609.03499*, 2016.

[12] Bram van Leer. Towards the ultimate conservative difference scheme V. A second–order sequel to Godunov's method. *Journal of Computational Physics*, 32(1):101–136, 1979.

[13] Sergei K. Godunov. A difference method for numerical calculation of discontinuous solutions of the equations of hydrodynamics. *Matematicheskii Sbornik*, 47(3):271–306, 1959.

[14] Randall J. LeVeque. *Finite Volume Methods for Hyperbolic Problems*. Cambridge University Press, 2002.

[15] Ami Harten, Peter D. Lax, and Bram van Leer. On upstream differencing and Godunov–type schemes for hyperbolic conservation laws. *SIAM Review*, 25(1):35–61, 1983.

[16] Sergei N. Kružkov. First–order quasilinear equations in several independent variables. *Mathematics of the USSR–Sbornik*, 10(2):217–243, 1970.

[17] Michael G. Crandall and Andrew Majda. Monotone difference approximations of scalar conservation laws. *Mathematics of Computation*, 34(149):1–21, 1980.

[18] Kurt Hornik. Approximation capabilities of multilayer feedforward networks. *Neural Networks*, 4(2):251–257, 1991.

[19] Jay P. Boris and David L. Book. Flux–corrected transport. I. SHASTA, a fluid transport algorithm that works. *Journal of Computational Physics*, 11(1):38–69, 1973.

[20] Steven T. Zalesak. Fully multidimensional flux–corrected transport algorithms for fluids. *Journal of Computational Physics*, 31(3):335–362, 1979.

[21] Antony Jameson. Analysis and design of numerical schemes for gas dynamics 1: artificial diffusion, upwind schemes, explicit difference schemes. *International Journal of Computational Fluid Dynamics*, 4(3–4):171–218, 1995.

[22] R. Borges, M. Carmona, B. Costa, and W. S. Don. An improved weighted essentially non–oscillatory scheme for hyperbolic conservation laws. *Journal of Computational Physics*, 227(6):3191–3211, 2008.

A Proofs of structural invariants

Proof of Theorem 4. Summing the update rule (1) over $i = 0, \dots, L-1$ gives

$$\sum_i p^{k+1}(i) = \sum_i p^k(i) + \lambda \sum_i (F_{i-1/2} - F_{i+1/2}) + \sigma \sum_i S_i.$$

The flux sum telescopes: $\sum_{i=0}^{L-1}(F_{i-1/2} - F_{i+1/2}) = F_{-1/2} - F_{L-1/2} = 0$ by the zero–flux boundary condition. If the source is conservative ($\sum_i S_i = 0$) or absent, the third term vanishes. Hence $\sum_i p^{k+1}(i) = \sum_i p^k(i)$, and by induction $\sum_i p^K(i) = \sum_i p^0(i)$.

For DTFN–B, the bidirectional merge $m_{\text{out}} = \alpha \odot m_{\text{fwd}} + (1-\alpha) \odot m_{\text{bwd}}$ preserves the sum because $\sum_i m_{\text{out}}(i) = \alpha \sum_i m_{\text{fwd}}(i) + (1-\alpha) \sum_i m_{\text{bwd}}(i) = \alpha \sum_i p^0 + (1-\alpha) \sum_i p^0 = \sum_i p^0$, using that m_{fwd} and m_{bwd} each conserve mass. (The equality holds channel–wise when α is a per–channel vector.)

For DTFN–MUSCL, the slope–limited reconstruction preserves the cell average: the face values $p^{\text{L}}_{i+1/2}$ and $p^{\text{R}}_{i+1/2}$ are constructed so that $\frac{1}{2}(p^{\text{L}}_{i+1/2} + p^{\text{R}}_{i+1/2}) = p_i$ when the slope σ_i is symmetric. The Van Leer limiter is conservative in this sense, so the telescoping argument applies. □ □

Proof of Theorem 5. The positivity–preserving step computes $\lambda_{\text{safe}} = \min(\lambda, \min_{i:\mathrm{d}m_i<0} \frac{p_i - \varepsilon}{-\mathrm{d}m_i})$. For any cell i with $\mathrm{d}m_i \geq 0$, $p_i^{k+1} = p_i^k + \lambda_{\text{safe}}\, \mathrm{d}m_i \geq p_i^k \geq \varepsilon$. For any cell i with $\mathrm{d}m_i < 0$, the definition of λ_{safe} ensures $\lambda_{\text{safe}} \leq \frac{p_i - \varepsilon}{-\mathrm{d}m_i}$, so $p_i^{k+1} = p_i^k + \lambda_{\text{safe}}\, \mathrm{d}m_i \geq p_i^k + \frac{p_i - \varepsilon}{-\mathrm{d}m_i}\, \mathrm{d}m_i = \varepsilon$. Hence $p_i^{k+1} \geq \varepsilon$ for all i. □ □

Proof of Theorem 6. Under the CFL condition $\lambda \in (0, 1)$ and monotonicity of the flux (non–decreasing in the left input, non–increasing in the right input, which is implied by our MonotoneFluxNet's $|W_{\text{left}}|$ parameterisation and Softplus output), the update $p_i^{k+1} = p_i^k + \lambda(F_{i-1/2} - F_{i+1/2})$ can be written as a convex combination of $\{p_{i-1}^k, p_i^k, p_{i+1}^k\}$ (Harten–Lax–van Leer monotonicity [15]). A convex combination preserves the min–max range: $\min_j p^k(j) \leq p_i^{k+1} \leq \max_j p^k(j)$ for all i. By induction, the range is preserved for all k.

For DTFN–FULL with source term S_i, the update becomes $p_i^{k+1} = (\text{convex combination}) + \lambda\, S_i$, so the range expands by at most $\lambda \max_i |S_i| \leq \sigma_{\max}$ per step (using $\lambda < 1$). Summing over k steps gives the modified bound $\min_i p^0 - k\sigma_{\max} \leq p_i^k \leq \max_i p^0 + k\sigma_{\max}$, which is the stated source–augmented maximum principle. □ □

Proof sketch of Theorem 7. The Van Leer limiter is TVD [12, 14]. When combined with a monotone flux and CFL $\lambda \in (0, 1)$, the MUSCL scheme inherits the TVD property from the underlying first–order scheme [15]. In particular, the total variation $\mathrm{TV}(p^{k+1}) = \sum_i |p_{i+1}^{k+1} - p_i^{k+1}|$ is bounded by $\mathrm{TV}(p^k)$. □ □

Proof sketch of Theorem 8. The DTFN update is *local*: p_i^{k+1} depends only on $\{p_{i-1}^k, p_i^k, p_{i+1}^k\}$ through a parameter–shared flux network. Consequently, the function f_θ mapping a length–L input to its output factorises as a composition of position–independent local operators. For any context–free grammar whose local statistics (the distribution of n–grams for n bounded by the receptive field K) are position–invariant, the loss on a sequence of length L_{test} equals the loss on a sequence of length L_{train} as long as both have the same local statistics. Dyck–k grammars are position–invariant in this sense: the next–token distribution depends only on the bracket depth and the last K tokens, not on absolute position. Hence DTFNs extrapolate exactly. A formal proof requires a position–invariance lemma and is deferred to a longer version. □ □

Proof of Theorem 9. We prove the single–step bound $\|p^{k+1} - \tilde{p}^{k+1}\|_1 \leq \|p^k - \tilde{p}^k\|_1$; the multi–step bound follows by induction. We give the argument for DTFN–U; the bidirectional case (DTFN–B) applies it to each stack and uses the convexity of the ℓ_1 norm.

Step 1 (monotonicity of the FluxNet). The DTFN–U update is $p_i^{k+1} = p_i^k + \lambda\big(F_L(p_{i-1}^k) - F_R(p_i^k)\big)$, where $F_L, F_R : \mathbb{R}_+^d \to \mathbb{R}_+^d$ are the left and right FluxNets. By construction (Softplus output, non–negative weights in the monotone form), both F_L and F_R are componentwise non–decreasing. Combined with CFL $\lambda \in (0, 1)$, the Crandall–Majda monotonicity framework [17] applies: the update operator $\mathcal{U} : p^k \mapsto p^{k+1}$ is monotone in the order sense ($p^k \leq \tilde{p}^k$ componentwise implies $\mathcal{U}(p^k) \leq \mathcal{U}(\tilde{p}^k)$) and conservative (Theorem 4).

Step 2 (ℓ_1 contraction via the Crandall–Majda lemma). For monotone conservative schemes, the Crandall–Majda ℓ_1–contraction lemma [17, Thm. 3.1] (equivalently, the Kružkov doubling argument adapted to the discrete case) gives

$$\sum_i |p_i^{k+1} - \tilde{p}_i^{k+1}| \;\le\; \sum_i |p_i^k - \tilde{p}_i^k|,$$

with the sum taken per channel. The key step in the lemma is to test the monotone update against $\operatorname{sign}(p_i^k - \tilde{p}_i^k)$ and use the telescoping of the conservative flux, which yields the contraction *without* requiring the convex–combination coefficients to be state–independent. This is essential for the neural setting, where the FluxNet's effective coefficients depend on the local state. Summing over channels gives the full $\|p^{k+1} - \tilde{p}^{k+1}\|_1 \le \|p^k - \tilde{p}^k\|_1$ bound.

Step 3 (from ℓ_1 to ℓ_∞). For an ℓ_∞–bounded perturbation $\|\delta^0\|_\infty \le \varepsilon$, we have $\|\delta^0\|_1 \le Ld\,\varepsilon$ (a factor of 2 arises if perturbations are signed), hence $\|\delta^K\|_1 \le Ld\,\varepsilon$. Applying the discrete maximum principle (Theorem 6) to the difference signal δ^k – which itself obeys a monotone update – yields the per–token bound $\|\delta^K\|_\infty \le \|\delta^0\|_\infty \le \varepsilon$. Thus the DTFN forward map is 1–Lipschitz in both ℓ_1 and ℓ_∞. □ □

Proof of Theorem 10. Linearise the update $p^{k+1} = p^k + \lambda\, D(p^k)$ about a fixed point p^*, where $D(p^k) = F_{i-1/2}(p^k) - F_{i+1/2}(p^k)$ is the discrete divergence. The Jacobian of the linearised update is $I + \lambda\, J_D$, where $J_D = \partial D/\partial p$ has spectral radius $\rho(J_D) \le 2\rho(J_F)$ (the divergence differences two flux Jacobians). The linearised recurrence is stable in ℓ_2 if $|1 + \lambda\mu| \le 1$ for every eigenvalue μ of J_D. For the monotone–flux parameterisation, J_F has eigenvalues in $[-1, 1]$ (Softplus output bounded in $[0, \infty)$ with bounded weights, and the monotone–form $|W_{\text{left}}|$ parameterisation ensures the spectral bound), so $\mu \in [-2, 0]$ and the condition becomes $|1-2\lambda| \le 1$, i.e. $\lambda \le 1$. The sigmoid parameterisation $\lambda = \sigma(\beta) \in (0, 1)$ certifies this by construction, and the linearised ℓ_2 norm is non–increasing: $\|p^{k+1}\|_2 \le \|I + \lambda J_D\|_2\, \|p^k\|_2 \le \|p^k\|_2$. □ □

Proof of Theorem 12. The DTFN update with shared parameters θ and per–step CFL $\lambda = T/K$ is

$$p_i^{k+1} = p_i^k + \tfrac{T}{K}\big(F(p_{i-1}^k, p_i^k; \theta) - F(p_i^k, p_{i+1}^k; \theta)\big),$$

which is exactly the first–order conservative finite–difference discretisation of $\partial_t p + \partial_x F(p; \theta) = 0$ on a uniform grid with mesh width h in x and time step $\Delta t = T/K = \lambda h$ in t. By the Crandall–Majda monotonicity theorem [17], a monotone, conservative, consistent scheme converges to the unique Kružkov entropy solution of the scalar conservation law as $h, \Delta t \to 0$ provided the CFL condition $\lambda\rho(J_F) \le 1$ holds. The MonotoneFluxNet is constructed to be monotone (Section 3), conservative (telescoping fluxes, Theorem 4), and consistent (the $v \cdot$left term recovers the upwind flux at initialisation, and the Softplus MLP perturbs it continuously). The CFL condition is certified by Theorem 10. Hence $p_i^k \to p(ih, k\Delta t)$ in L^1 as $K \to \infty$.

For the MUSCL variant, the same argument applies with the second–order Lax–Wendroff–type convergence rate $\mathcal{O}(h^2)$, provided the Van Leer limiter is used (the scheme remains TVD, Theorem 7) and the CFL condition holds [14]. □ □

Proof sketch of Theorem 13. The proof has three parts.

Step 1 (FluxNet universality). The MonotoneFluxNet $F(\text{left}, \text{right}; \theta)$ is a two–input, single–output MLP with Softplus output. By the Cybenko–Hornik universal approximation theorem [18], a single hidden layer of sufficient width can approximate any continuous function on a compact set to arbitrary accuracy. The monotonicity constraints ($|W_{\text{left}}|$, $-|W_{\text{right}}|$, $|W_{\text{out}}|$) restrict the hypothesis class to monotone fluxes, but any *monotone* continuous flux can be approximated: the monotone cone is dense in itself, and the Softplus–output MLP with signed weights can realise any non–negative continuous function on a compact set (Hornik's theorem applied to the non–negative cone).

Step 2 (Stacking realises any local map). A single FV update couples cell i to $\{i-1, i, i+1\}$; K stacked updates couple i to $\{i - K, \ldots, i + K\}$, giving a receptive field of $2K + 1$. Conversely, any translation–equivariant local map with receptive field $r = 2K + 1$ can be written as a sequence of K pairwise couplings (a product of shift–invariant kernels), each realisable by a MonotoneFluxNet. This is the discrete analogue of the result that any local causal operator can be decomposed into a product of infinitesimal couplings.

Step 3 (Decoder universality). The per–token decoder $\text{dec} : \mathbb{R}^d \to \mathbb{R}^V$ is a 2–layer MLP with GELU, which is a universal approximator [18] on the per–token mass.

Combining the three steps: for any continuous local translation–equivariant $\mathcal{F}$ and any $\varepsilon > 0$, choose K so that the receptive field $2K + 1 \geq r$, choose the FluxNet width to approximate the per–coupling flux to $\varepsilon/3$, and choose the decoder width to approximate the per–token readout to $\varepsilon/3$. The triangle inequality gives the final ε bound. □ □

A.1 Proof of Theorem 3 (FCT conservation and positivity)

Proof. **(a) Conservation.** The FCT update is $p_i^{k+1} = p_i^k + \lambda(F_{i-1/2}^{\mathrm{FCT}} - F_{i+1/2}^{\mathrm{FCT}})$, where $F^{\mathrm{FCT}} = F_{\mathrm{low}} + \phi \odot A$ and $A = F_{\mathrm{high}} - F_{\mathrm{low}}$. Both F_{low} and A are fluxes (they live on face indices), so summing the update over all cells:

$$\begin{aligned}\sum_i p_i^{k+1} &= \sum_i p_i^k + \lambda \sum_i (F_{i-1/2}^{\mathrm{FCT}} - F_{i+1/2}^{\mathrm{FCT}}) \\ &= \sum_i p_i^k + \lambda \Big[\sum_i F_{\mathrm{low},i-1/2} - \sum_i F_{\mathrm{low},i+1/2} + \sum_i (\phi_{i-1/2} A_{i-1/2}) - \sum_i (\phi_{i+1/2} A_{i+1/2}) \Big].\end{aligned}$$

Each bracketed sum telescopes to zero (with zero–flux boundary conditions $F_{-1/2} = F_{L-1/2} = 0$), so $\sum_i p_i^{k+1} = \sum_i p_i^k$.

(b) Positivity. Let $p_{\mathrm{low}} = p + \lambda(F_{\mathrm{low}}^{\mathrm{in}} - F_{\mathrm{low}}^{\mathrm{out}})$ be the low–order update, with $p_{\mathrm{low}} \geq 0$ by the monotonicity of F_{low} and the standard DTFN positivity step. The FCT update can be written as $p^{k+1} = p_{\mathrm{low}} + \lambda\phi \odot (A^{\mathrm{in}} - A^{\mathrm{out}})$, where $A_i^{\mathrm{in}} = A_{i-1/2}$ and $A_i^{\mathrm{out}} = A_{i+1/2}$. Let $\Delta A_i = A_i^{\mathrm{in}} - A_i^{\mathrm{out}}$ denote the net anti–diffusive contribution to cell i. Define the Zalesak coefficient

$$\alpha_i = \min\left(1,\ \frac{p_{\max,i} - p_{\mathrm{low},i}}{\max(\Delta A_i, 0) + \varepsilon},\ \frac{p_{\mathrm{low},i} - p_{\min,i}}{\max(-\Delta A_i, 0) + \varepsilon}\right),$$

where $p_{\max,i}, p_{\min,i}$ are the local extrema of p_{low} in the stencil of cell i. If $\phi_{i\pm1/2} \leq \min(\alpha_i, \alpha_{i+1})$ for every face adjacent to cell i, then $p_{\mathrm{low},i} + \lambda\phi\Delta A_i \in [p_{\min,i}, p_{\max,i}] \subseteq [0, \infty)$, so $p_i^{k+1} \geq 0$. Our implementation enforces this by computing α_i for each cell and taking $\phi_{i+1/2} \leftarrow \min(\phi_{i+1/2}, \alpha_i, \alpha_{i+1})$ before the update.

(c) High–order recovery. In smooth regions, the Jameson smoothness sensor $\sigma_i \to 0$ (it is $\mathcal{O}(\Delta x^2)$ for smooth p), so the analytic limiter $\phi = 1 - \sigma \to 1$ and $F^{\mathrm{FCT}} = F_{\mathrm{low}} + A = F_{\mathrm{high}}$, recovering the high–order flux. For the learned limiter, if $\mathrm{LimiterNet}_\psi$ outputs $\phi \to 1$ on smooth inputs (which is the supervised target), the same recovery holds. □

B Implementation details

Our PyTorch implementation comprises approximately 300 lines of code in `dtfn_core.py` and 350 lines of experiment scripts in `dtfn_experiments_v2.py`. Key hyperparameters:

- Embedding dimension: $d = 64$.
- Number of update steps: $K = 3$ (DTFN–U/B/MUSCL), $K = 4$ (DTFN–FULL).
- FluxNet hidden width: 128.
- Initial CFL logit: 0 (so $\lambda = 0.5$ at initialisation).
- Positivity floor: $\varepsilon = 10^{-6}$.
- Optimiser: Adam, learning rate 10^{-3}, batch size 32.
- Gradient clipping: $\|\nabla\|_2 \leq 1$.
- Epochs: 5 (Dyck–1), 8 (Dyck–2), 5 (extrap, copy).

All experiments were run on an Apple M4 CPU with 16 GB RAM. Training the full Dyck–1 multi–seed comparison (6 models × 5 seeds × 5 epochs) takes approximately 25 minutes. The scaling experiments (Figure 6) use a batch size of 8 and average over 5 forward passes after 2 warmup iterations.

B.1 CPU wall–clock scaling reference

For completeness, Table 17 reports the CPU (M4, MPS backend) wall–clock times used as the reference for Section 4.16. The linear scaling trend is backend–independent because the finite–volume update is $\mathcal{O}(LKd)$ with no L^2 attention term; an Ascend ASI3 NPU port would preserve the trend and improve absolute throughput.

Table 17: CPU (Apple M4, MPS backend) wall–clock time per forward pass (ms), batch size 1, $d = 64$, $K = 3$, averaged over 5 passes after 2 warmup iterations. Note: $L = 256$ is faster than $L = 512$ due to fixed–overhead kernel launch costs that dominate at small L.

L	256	512	1024	2048	4096	8192	16384
DTFN–U	4.14	3.38	3.82	4.28	5.75	10.18	18.55
DTFN–MUSCL	3.34	3.75	4.05	4.76	6.66	12.75	21.84
DTFN–WENO5	5.33	6.48	10.57	14.94	23.65	39.37	69.53